

第5章 回溯法

学习要点

- 理解搜索算法 DFS BFS
- 理解回溯法的深度优先搜索策略

□ 掌握用回溯法解题的算法框架

- (1) 递归回溯
- (2) 迭代回溯
- (3) 子集树算法框架
- (4) 排列树算法框架

例子:

- (1) 0-1背包问题
- (2) 旅行售货员问题
- (3) n 后问题
- (4) 装载问题

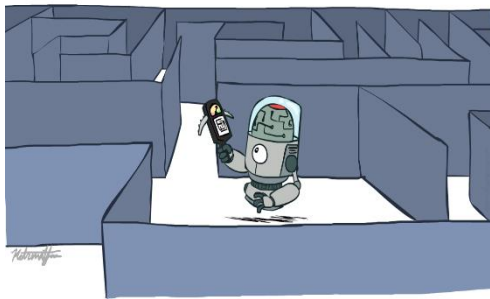
问题求解与搜索算法

问题求解涉及到两个方面：

- 1) **问题的表示**
- 2) **求解的方法**

□ 搜索算法是一种**通用的问题求解方法**：首先把问题表示转换为一个状态空间图，然后设计特定的**图遍历方法**在状态空间中搜索问题的答案。

□ 为了提高搜索的效率，在遍历状态空间时**需要添加优化技术**，比如**剪枝策略**用于尽可能避免无效搜索，**启发式信息**用来加速朝目标状态逼近的速度。



状态空间图

一个问题用搜索算法求解时，往往需要把问题描述为状态空间图，包括以下要素：

状态 (State)： 是为描述某类不同事物间的差别而引入的一组最少变量的有序集合，其矢量形式为： $X = [x_1, x_2, \dots, x_n]$ ，其中每个分量 $x_i (i = 1, \dots, n)$ 称为状态分量。

操作符 (运算符)： 是指把一个状态转换为另外一个状态的操作或者运算。操作符可以是规划、数学运算和问题场景中行为等。

状态图： 如果把状态定义为图的结点，操作符定义为图的边，一个问题的全部可能状态则可以表示为一个图，即状态图。

路径： 通过操作符序列连接起来的状态图中的一个状态序列。

路径耗散函数： 定义在路径上的一个数值函数，它反映了一条路径的性能度量或者求解问题的代价。在求解最优化问题时，路径耗散函数往往与优化目标相关联。

状态空间图

状态空间图可以形式化地定义为一个四元组 (S, A, G, F)

- S表示问题的**初始状态**，它是搜索的起点。
- A是采取的**操作符集合**，初始状态和操作符隐含地定义了问题的状态图。
- G表示**目标测试**，它判断给定的状态是否为目标状态。它可以是表示目标状态的一个状态集合，也可以是一个判定函数。
- F代表**路径耗散函数**，它的定义需要具体问题具体分析。

搜索就是在状态空间图中从初始状态出发，执行特定的操作，试探地寻找目标状态的过程。当然，也可以从目标结点到初始结点反向进行。状态空间图中从初始状态到目标状态的路径则代表**问题的解**。**解的优劣**由路径耗散函数量度，最优解就是路径耗散函数值最小的路径。

状态空间图-传教士问题

【例1】在河的左岸有三个传教士、一条船和三个野人，传教士们想用这条船将所有的成员都运过河去，但是受到以下条件的限制：

- ① 教士和野人都会划船，但船一次最多只能装运两个；
 - ② 在任何岸边野人数目都不得超过传教士，否则传教士会遭遇危险：被野人攻击甚至吃掉。
- 此外，假定野人会服从任何一种过河安排，试设计出一个确保全部成员安全过河的计划。

1) 状态表示 确定问题的状态表示，以及每一个状态变量的值域。

渡河问题包括三类对象：传教士，野人和渡船，得三元组 $S = (m, c, b)$ ，其中：

- ✓ m 为左岸传教士数，有 $m = \{0, 1, 2, 3\}$ ；对应右岸的传教士数为 $3 - m$ 。
- ✓ c 为左岸的野人数，有 $c = \{0, 1, 2, 3\}$ ；对应右岸野人数为 $3 - c$ 。
- ✓ b 为左岸渡船数，有 $b = \{0, 1\}$ ，右岸的船数为 $1 - b$ 。

初始状态只有一个，即 $S_0 = (3, 3, 1)$ ，表示全部成员在河的左岸；目标状态也只有一个，即 $S_g = (0, 0, 0)$ ，表示全部成员从河左岸渡河完毕。

状态空间图-传教士问题

状态	(m, c, b)	状态	(m, c, b)	状态	(m, c, b)	状态	(m, c, b)
S_0	331	S_8	131	S_{16}	330	S_{24}	130
S_1	321	S_9	121	S_{17}	320	S_{25}	120
S_2	311	S_{10}	111	S_{18}	310	S_{26}	110
S_3	301	S_{11}	101	S_{19}	300	S_{27}	100
S_4	231	S_{12}	031	S_{20}	230	S_{28}	030
S_5	221	S_{13}	021	S_{21}	220	S_{29}	020
S_6	211	S_{14}	011	S_{22}	210	S_{30}	010
S_7	201	S_{15}	001	S_{23}	200	S_{31}	000

表中的状态并不全都是合法的状态，可以删除非法的状态，从而加速搜索过程。

2) 操作符集合

- 把船从左岸划向右岸定义为 L_{ij} 操作，第一下标 i 表示船载的传教士数，第二下标 j 表示船载的野人数。
- 从右岸将船划回左岸称之为 R_{ij} 操作，下标的定义同前。
- 则共有10种操作，操作集为

$$F = \{L_{01}, L_{10}, L_{11}, L_{02}, L_{20}, R_{01}, R_{10}, R_{11}, R_{02}, R_{20}\}$$

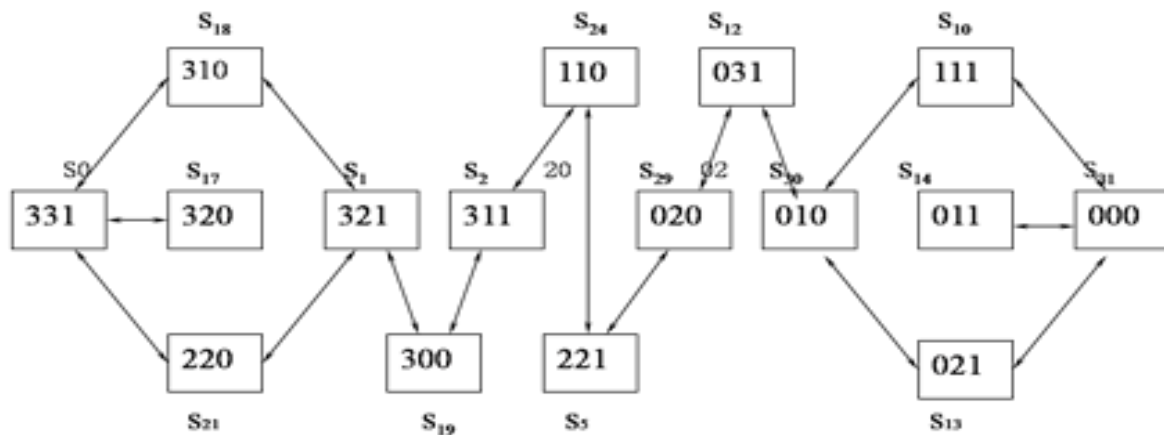
状态空间图-传教士问题

1) 状态表示: $S = (m, c, b)$

初始状态 $S_0 = (3, 3, 1)$, 目标状态 $S_g = (0, 0, 0)$

2) 操作符集合

$F = \{L_{01}, L_{10}, L_{11}, L_{02}, L_{20}, R_{01}, R_{10}, R_{11}, R_{02}, R_{20}\}$



状态空间图-8 数码问题

【例2】把左图的8数码排列通过空格的上下左右移动，转换为右图的8数码排列

□ 状态表示

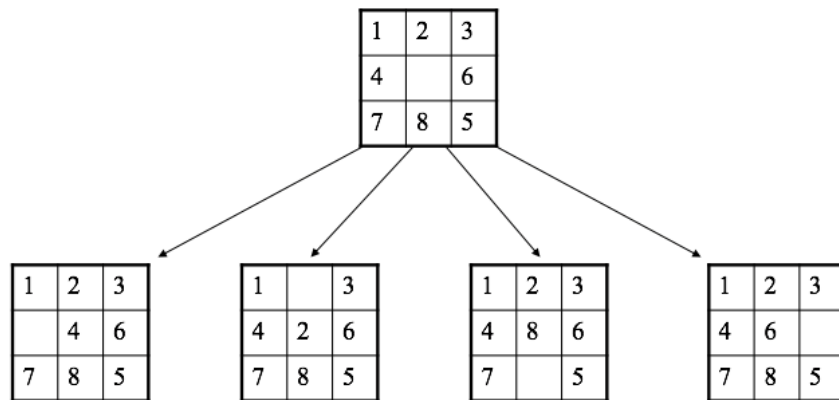
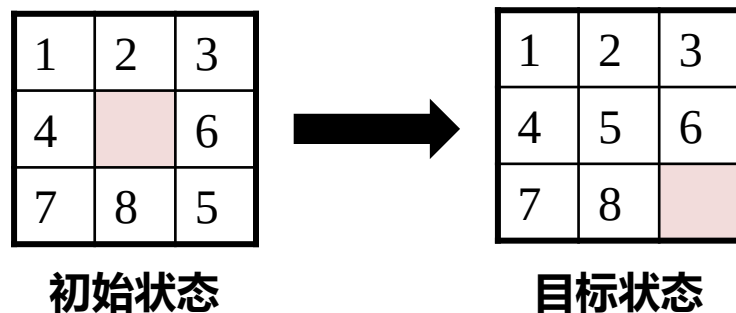
数字格与空格组成的排列

□ 操作符集合

空格向上、下、左和右移动

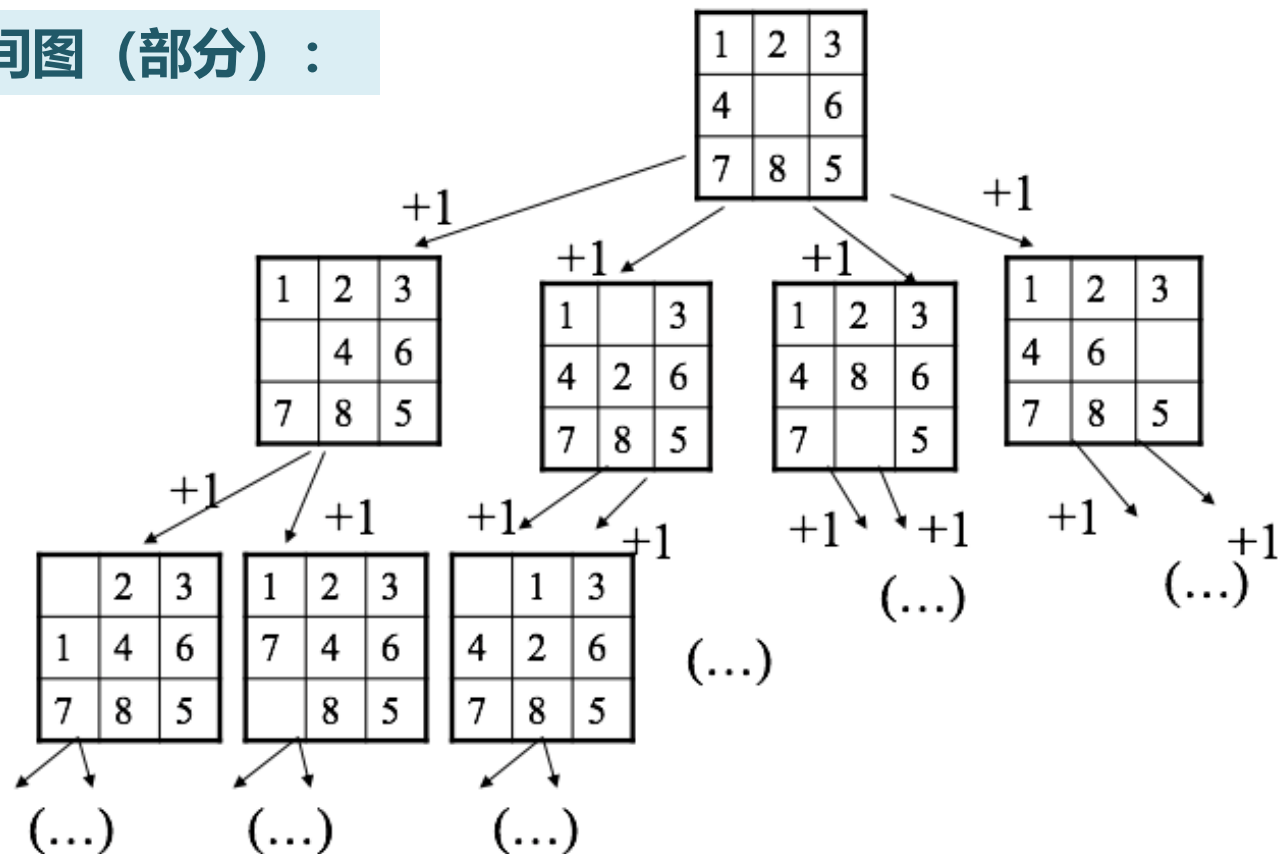
□ 路径耗散函数

每一次移动代价为 1，路径耗散等于移动次数



状态空间图-8 数码问题

状态空间图（部分）：



搜索算法类型

■ 基于枚举策略的通用搜索

- ✓ 深度优先搜索
- ✓ 广度优先搜索

■ 枚举 + 优化的通用搜索

- ✓ 回溯算法 = 深度优先搜索 + 剪枝策略
- ✓ 分支限界算法 = 广度优先搜索 + 剪枝策略

■ 启发式搜索，启发式搜索是一种基于规则的优化搜索算法。

■ 对抗搜索，DeepBlue，AlphaGO中的搜索

深度优先搜索 (DFS)

深度优先搜索 (Depth First Search, DFS) 是一种通用的图和树的遍历方法，给定图 $G = (V, E)$ ，深度优先搜索的基本思想为：

□ 初始化

任选一个结点 v 作为**源结点**，DFS访问源结点 v ，并将其标记为**已访问**过

□ 递归遍历

“能进则进，不进则退”

1. **扩展**结点 v ，即访问 v 的下一个**未访问**邻接结点 w ；
2. **递归**处理以 w 为根的所有子结点；
3. **退回**到结点 v ，继续扩展 v 的其它未曾访问的邻接结点，直到 v 的所有子结点均已访问过为止。

□ 循环处理

若图 G 中仍然存在未访问的结点，则另选一个未访问的结点作为**新的源结点**重复上述过程，直到图中所有结点均被访问过为止。

深度优先搜索 (DFS)

基于栈的DFS算法框架：

```
function DFS(problem, stack)           //stack初始化为空
    node = Make-Node(Initial-State[problem]); //生成初始结点
    stack ← Insert(node, stack);        //栈中插入初始结点
    do while (1)
        if stack == Empty
            return failure;             //没有搜索到目标状态
        node ← Remove-First(stack);     //取出栈顶元素
        visit(node);                    //访问栈顶元素
        if State[node] == Goal          //目标测试
            return Solution(node)      //搜索到目标状态
        sonNodes = Expend(node, problem); //扩展node
        stack ← Insert-All(sonNodes, stack); //全部未访问子结点加入栈
```

深度优先搜索 (DFS)

基于递归的DFS算法框架：

```
function DFS(problem, node)
  if State[node] == Goal/Failure    // 递归出口
    return Solution(node);
  else
    visit(node);                    // 访问当前结点
    for(iterator sonNode = Init(node); sonNode <= Last(node); sonNode++)
      if not Visited(sonNode)
        DFS(sonNode);              // 递归遍历其子树
  return;
```

注解：for循环用于扩展node的所有未访问子结点，并依次递归调用，其中Init表示node的第一个子结点，Last表示最后一个子结点。

DFS 通过队列 (Queue) 实现，遵循先进先出的规则。

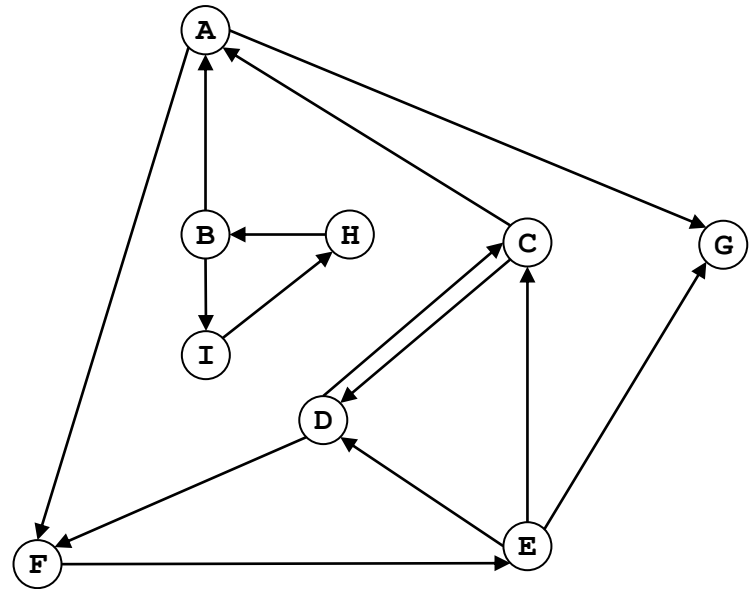
☒ A 错误

☐ B 正确

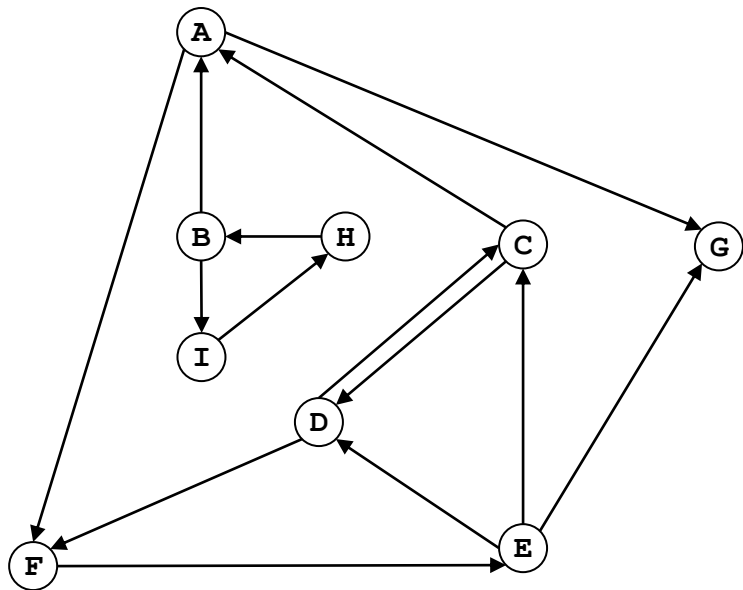
提交

DFS 演示

【例】给定一个有向图 $G = (V, E)$ ，如下图所示，给出该图深度优先搜索的一个访问序列。



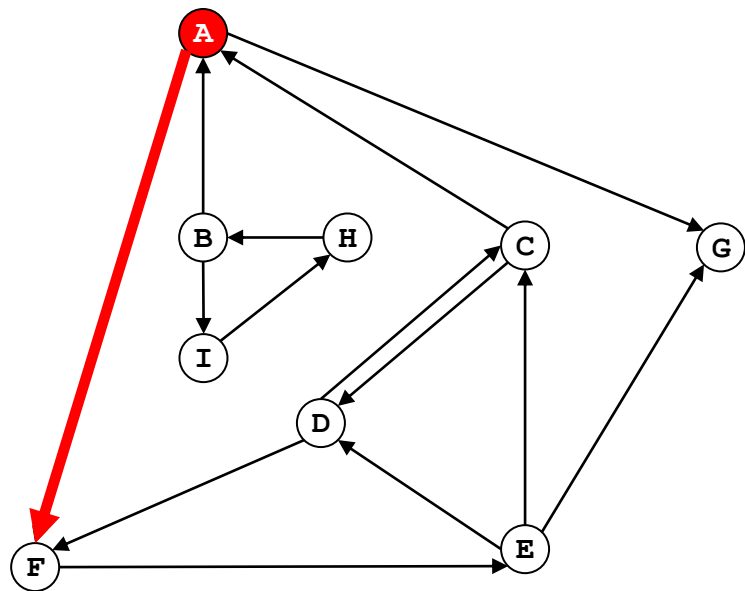
DFS 演示



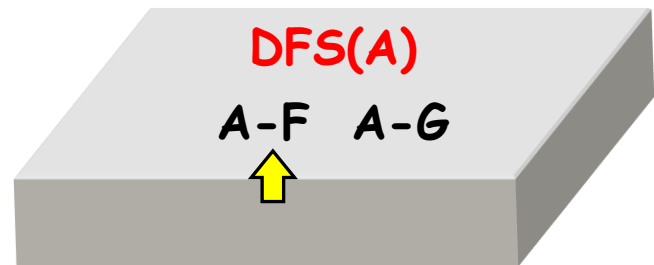
邻接表:

A:	F	G	
B:	A	I	
C:	A	D	
D:	C	F	
E:	C	D	G
F:	E		
G:			
H:	B		
I:	H		

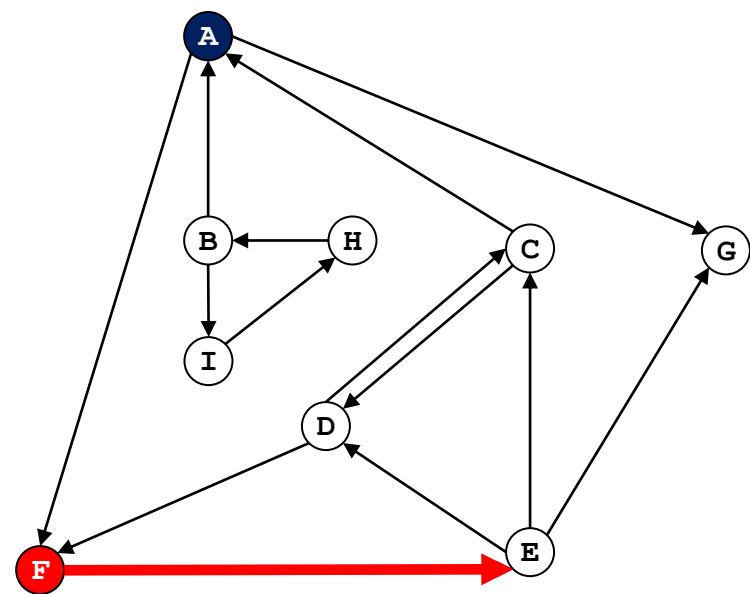
DFS 演示



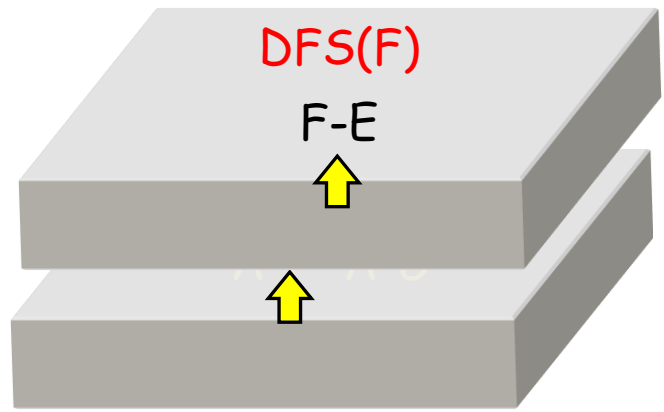
系统栈:



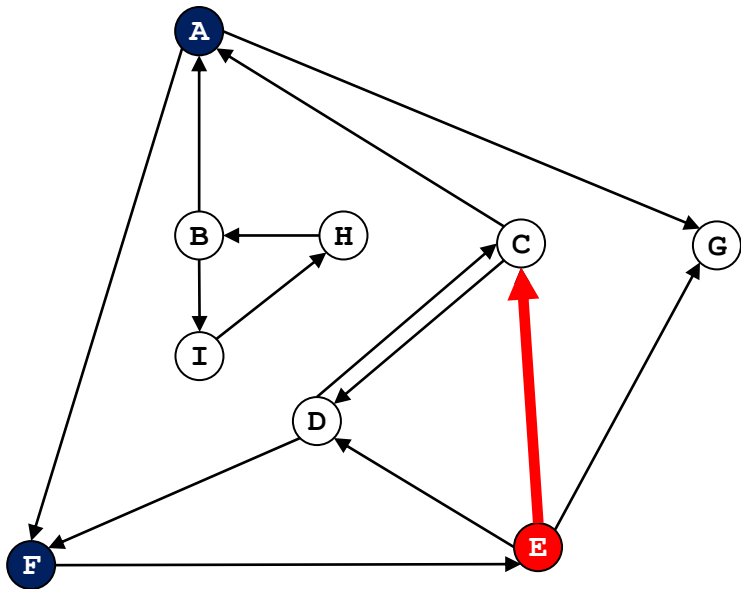
DFS 演示



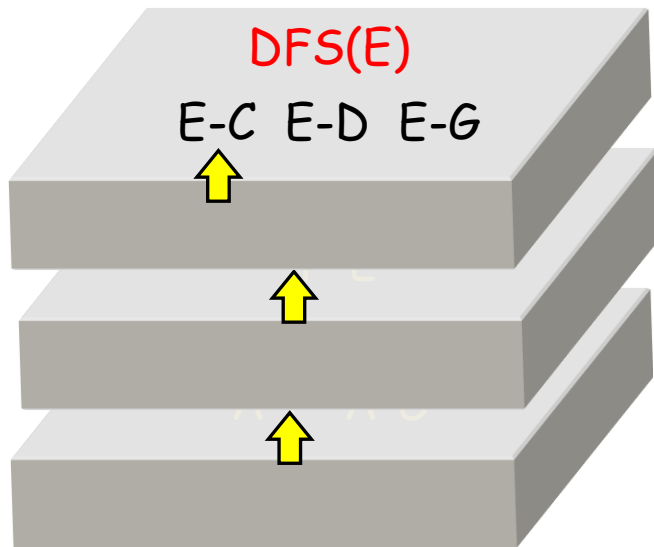
系统栈:



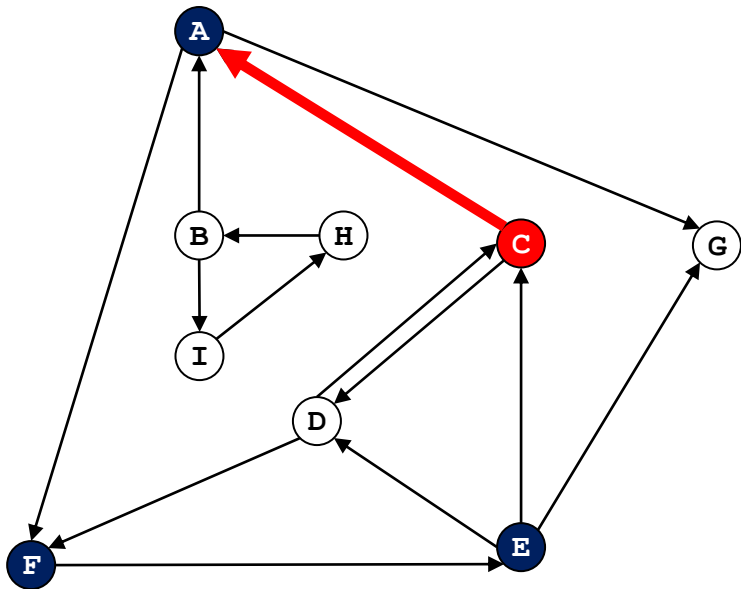
DFS 演示



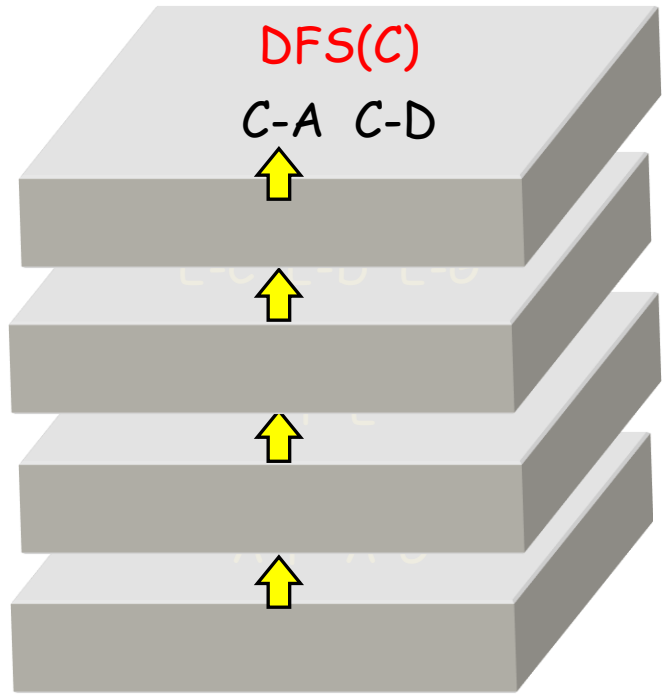
系统栈:



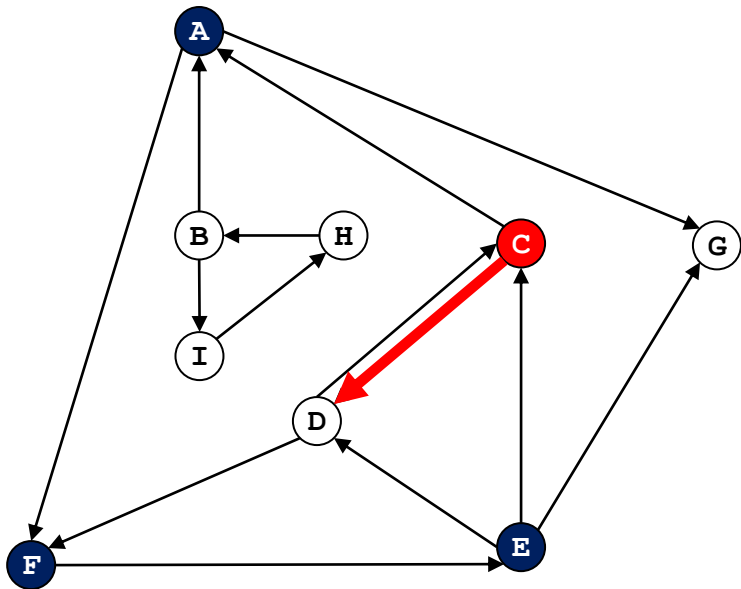
DFS 演示



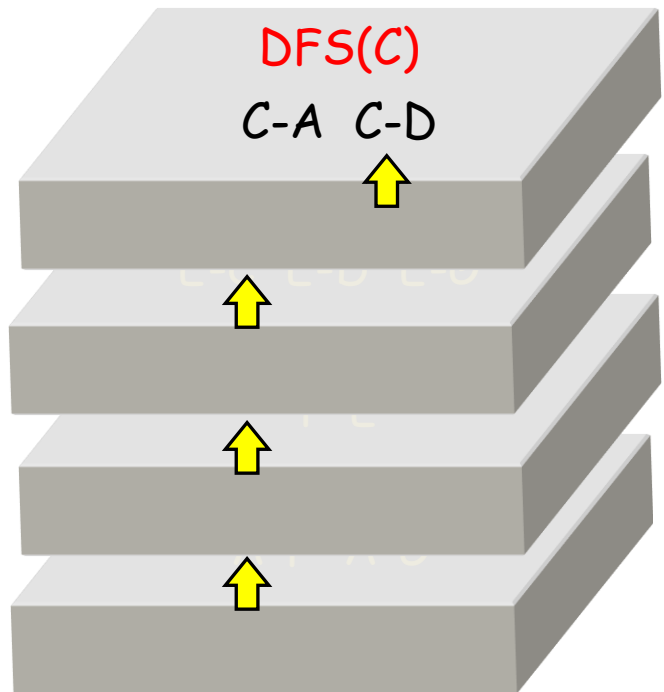
系统栈:



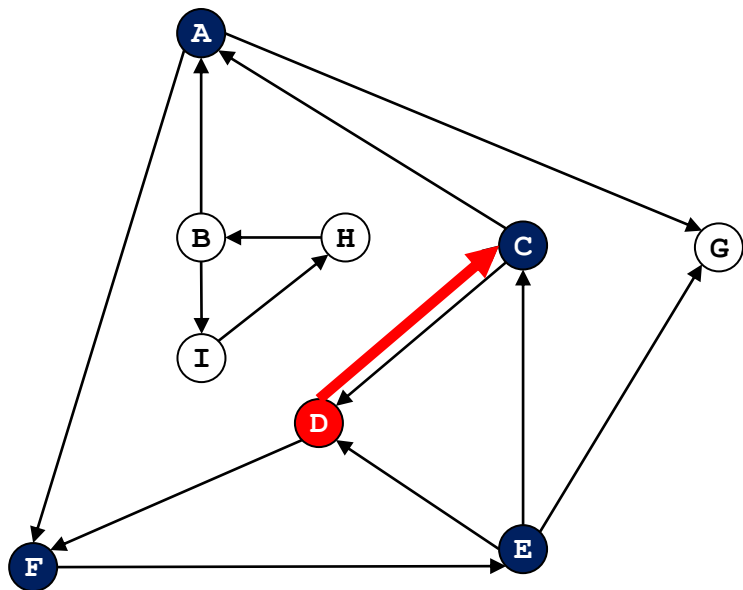
DFS 演示



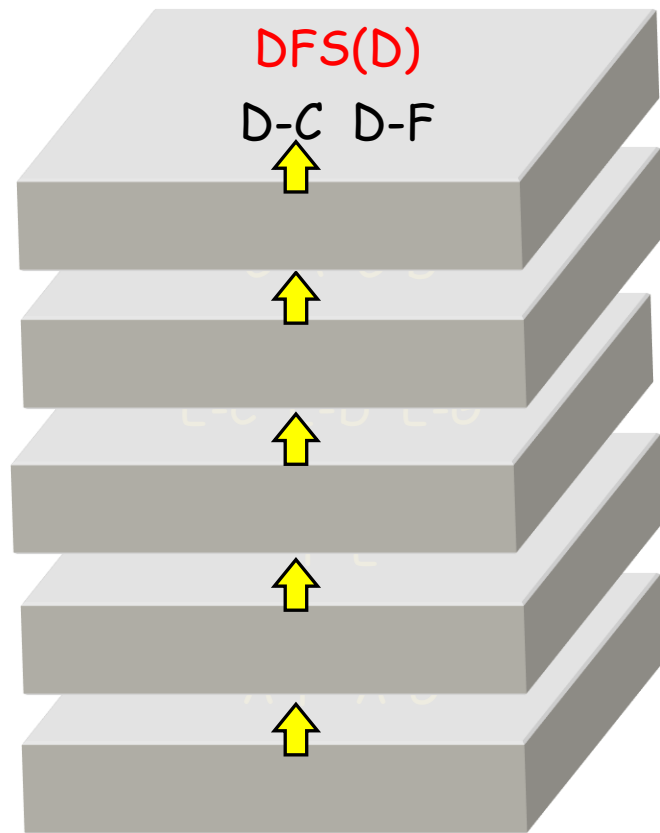
系统栈:



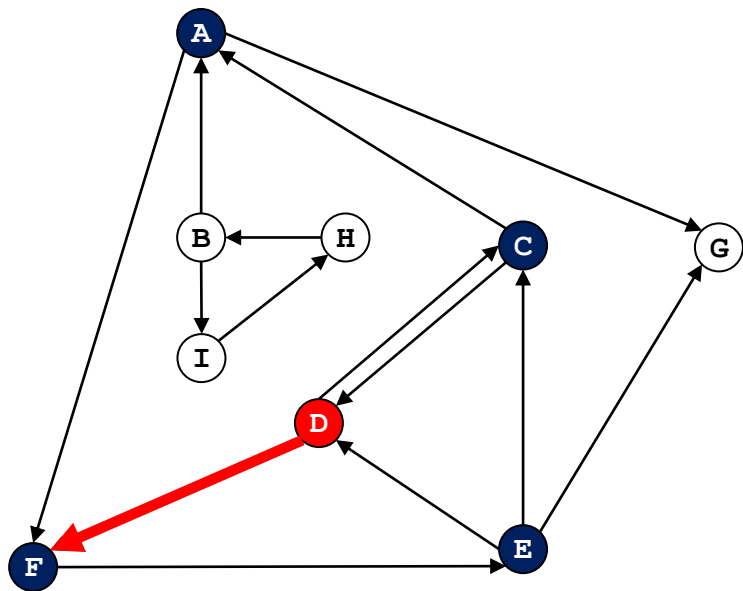
DFS 演示



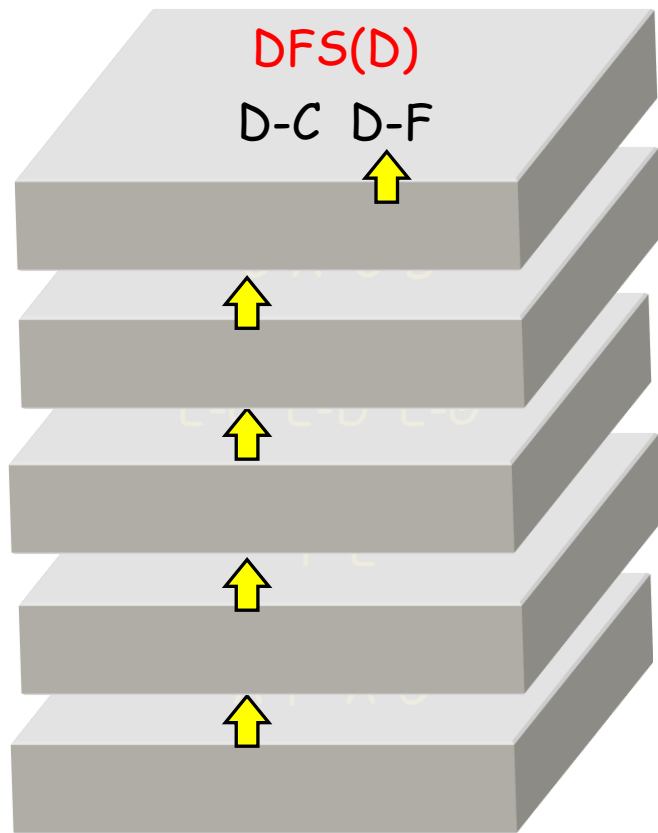
系统栈:



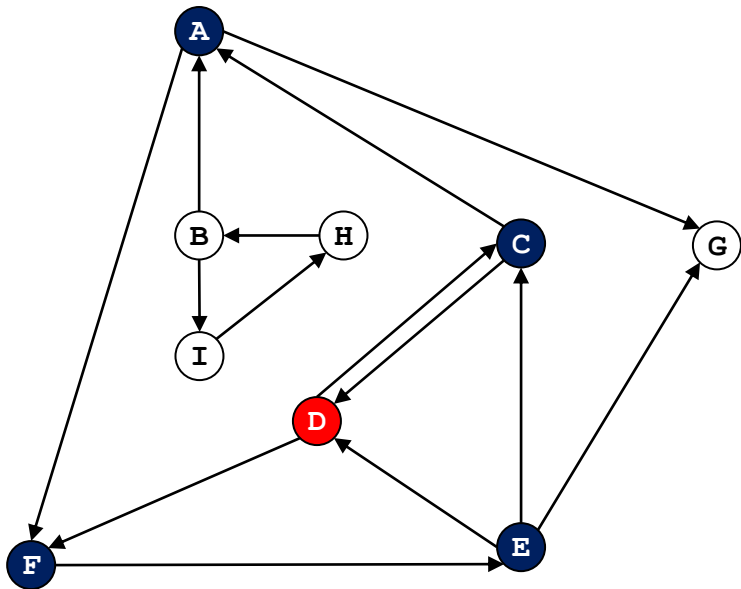
DFS 演示



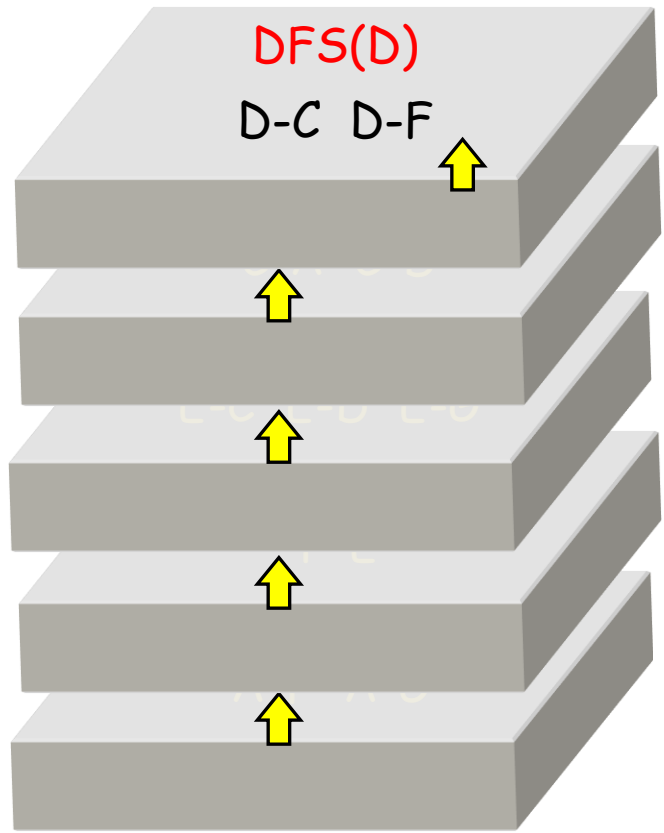
系统栈:



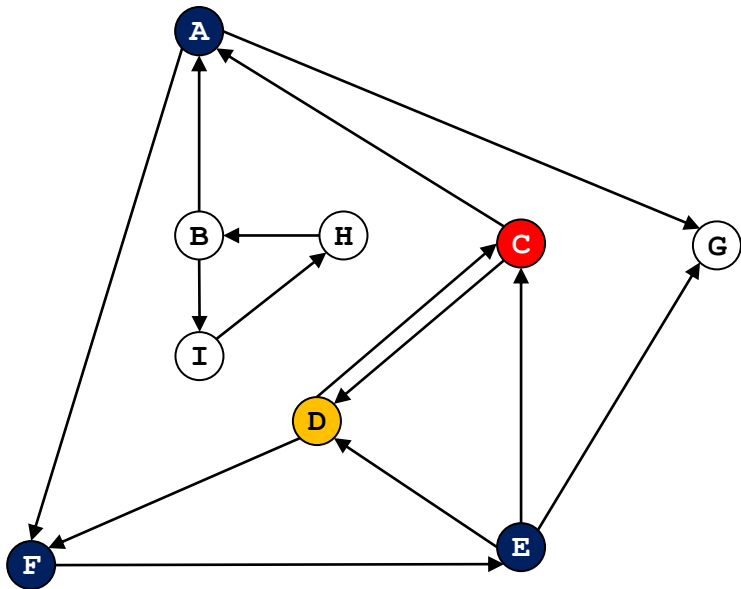
DFS 演示



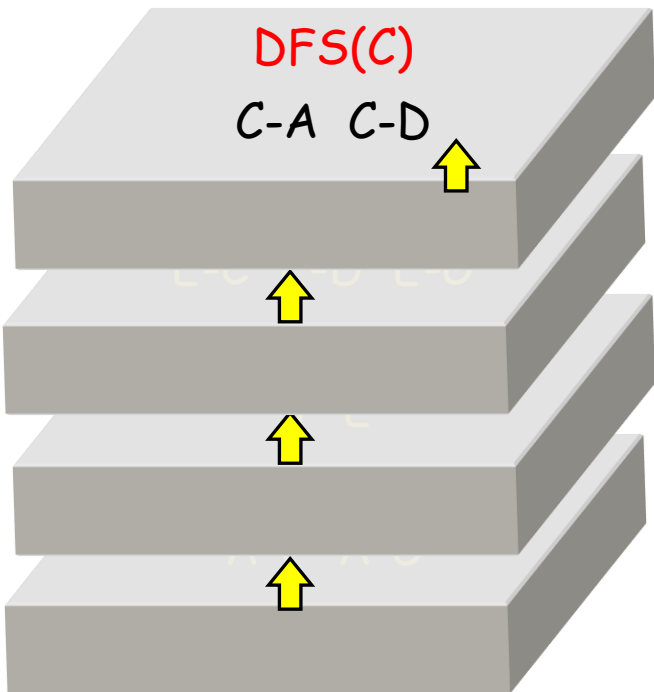
系统栈:



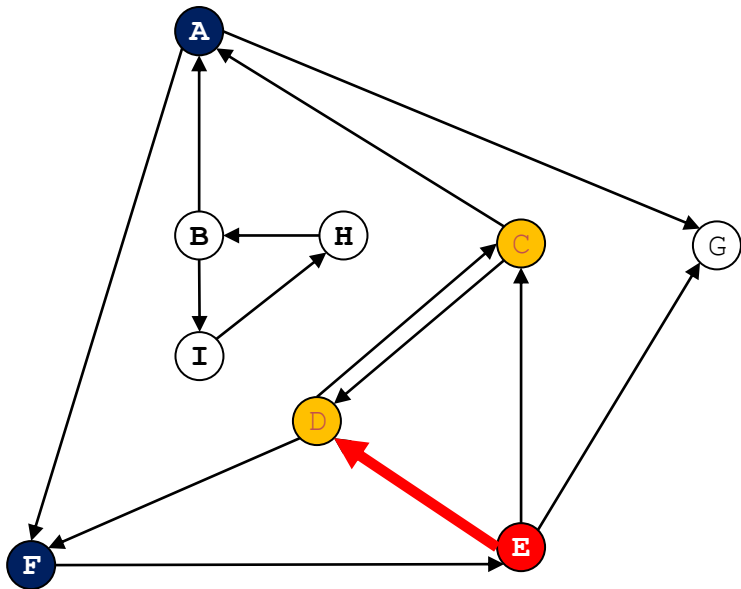
DFS 演示



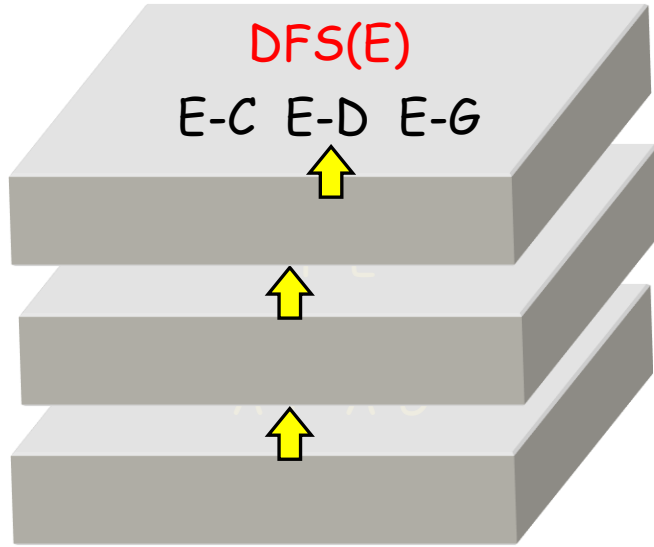
系统栈:



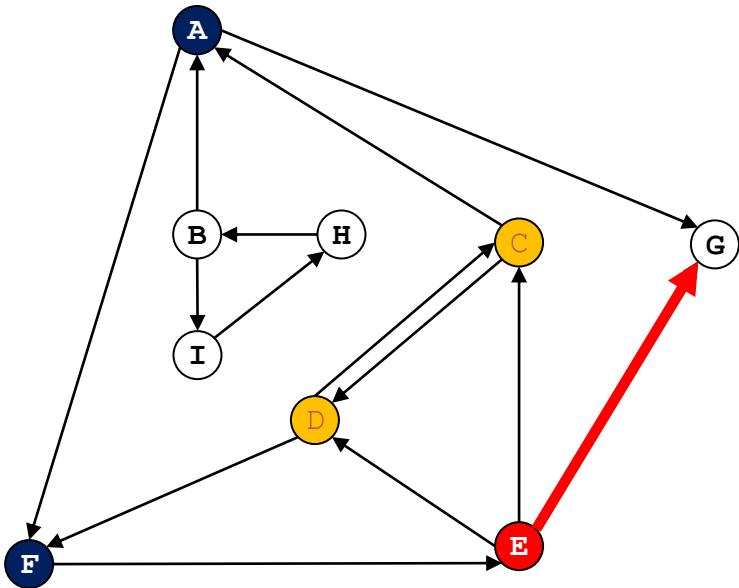
DFS 演示



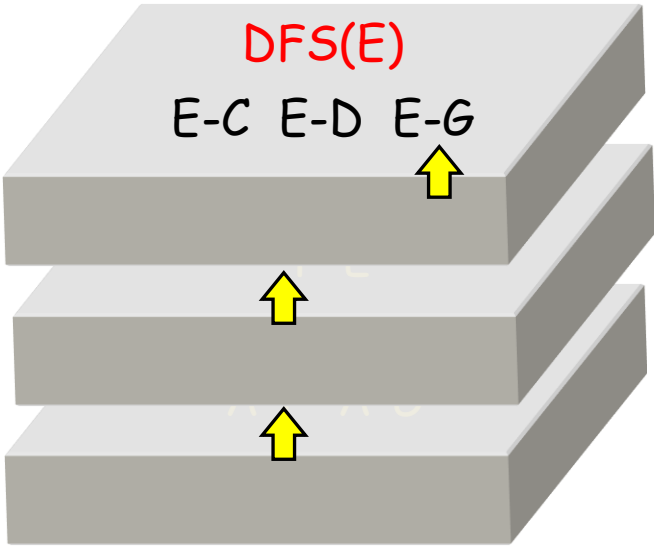
系统栈:



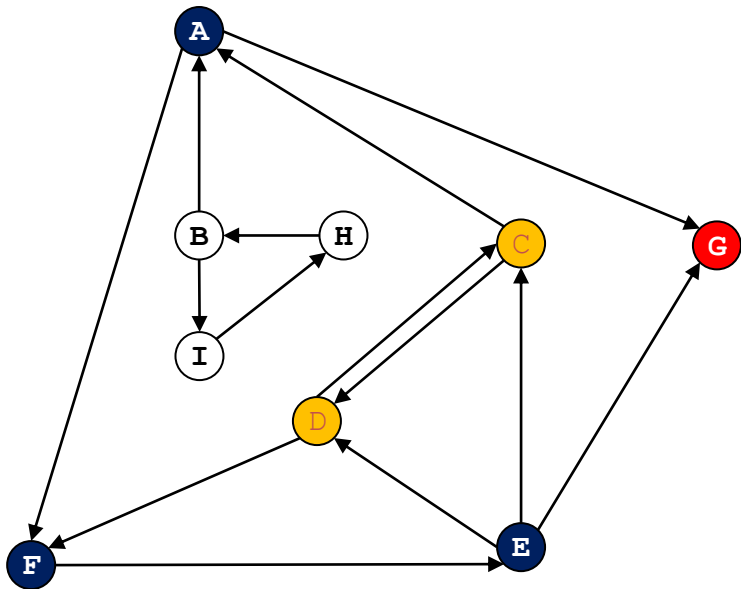
DFS 演示



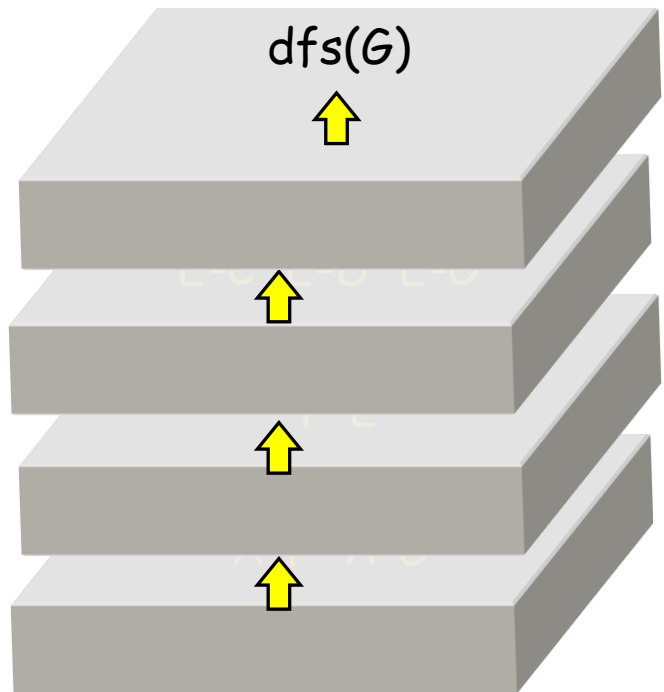
系统栈:



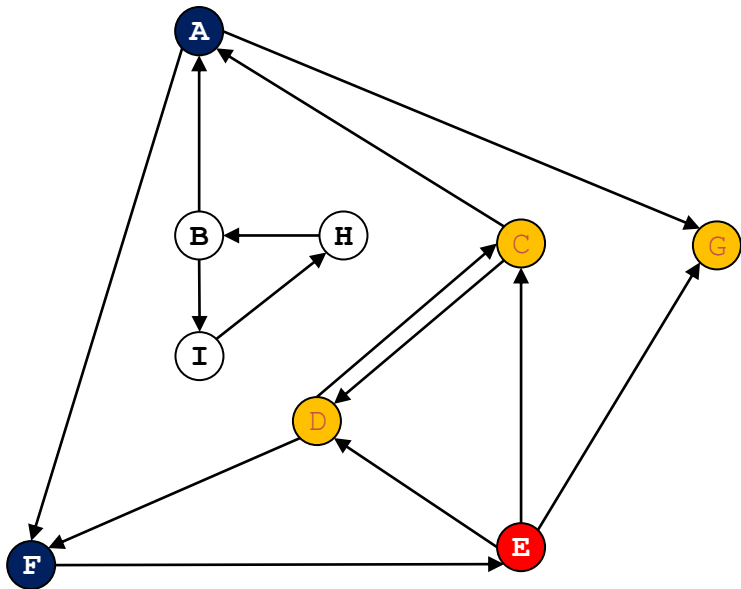
DFS 演示



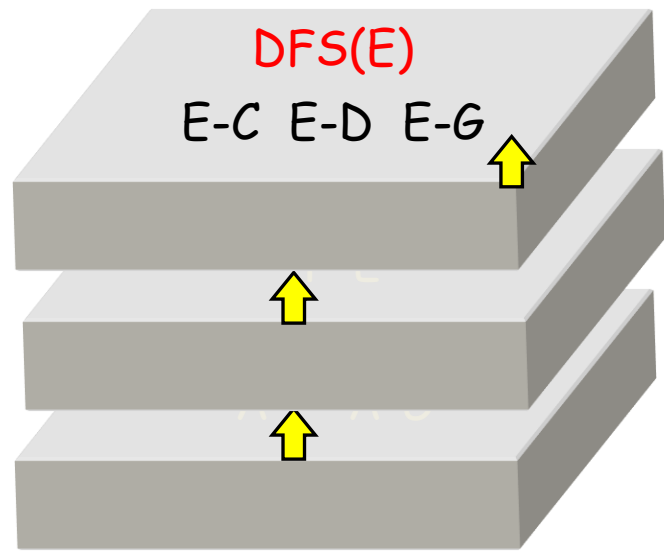
系统栈:



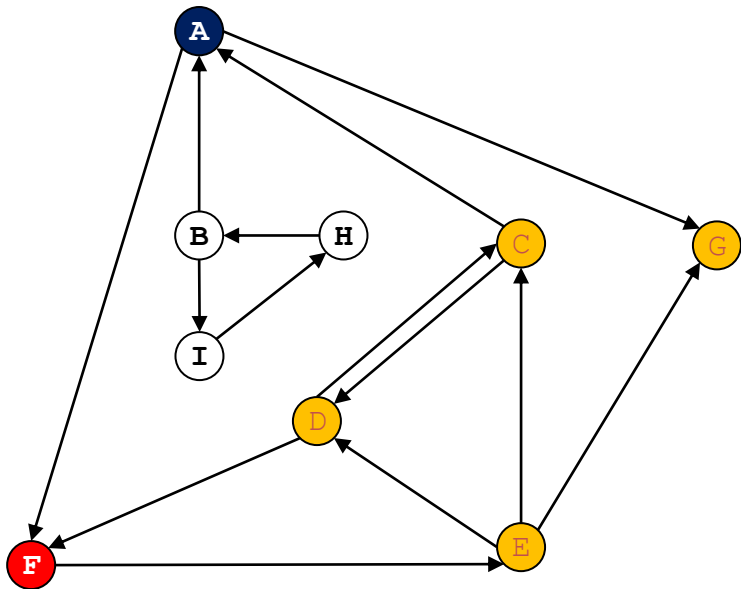
DFS 演示



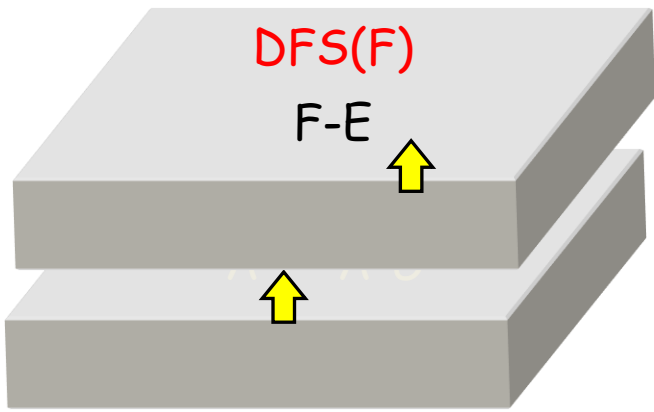
系统栈:



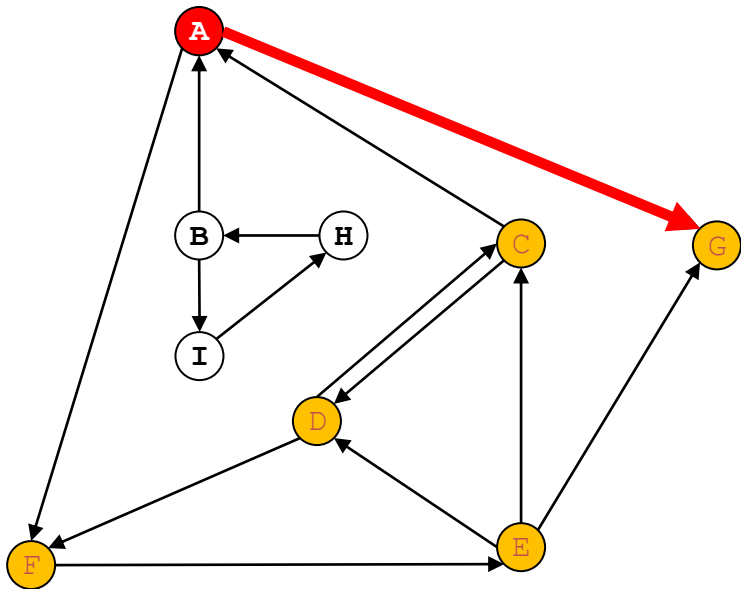
DFS 演示



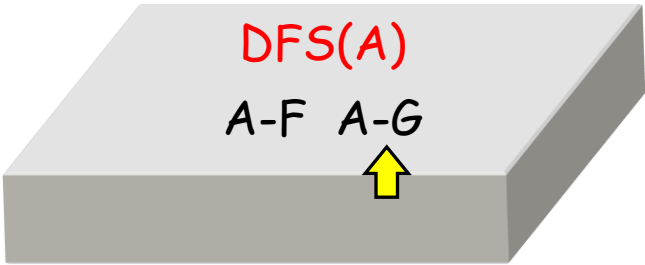
系统栈:



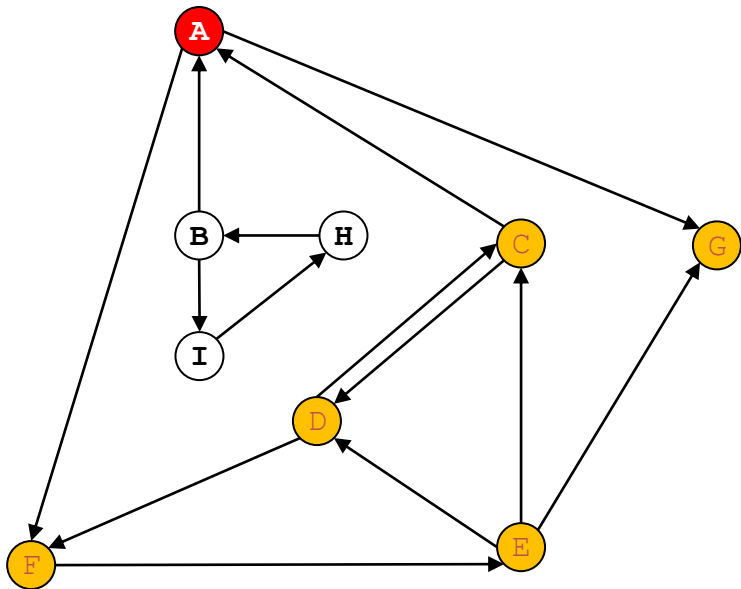
DFS 演示



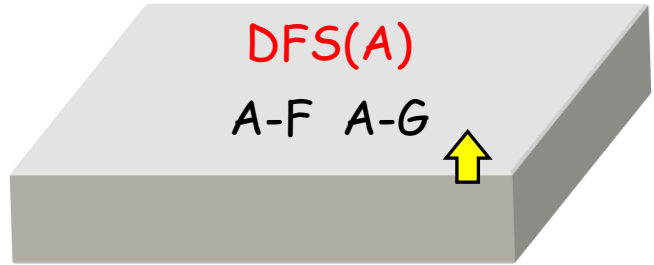
系统栈:



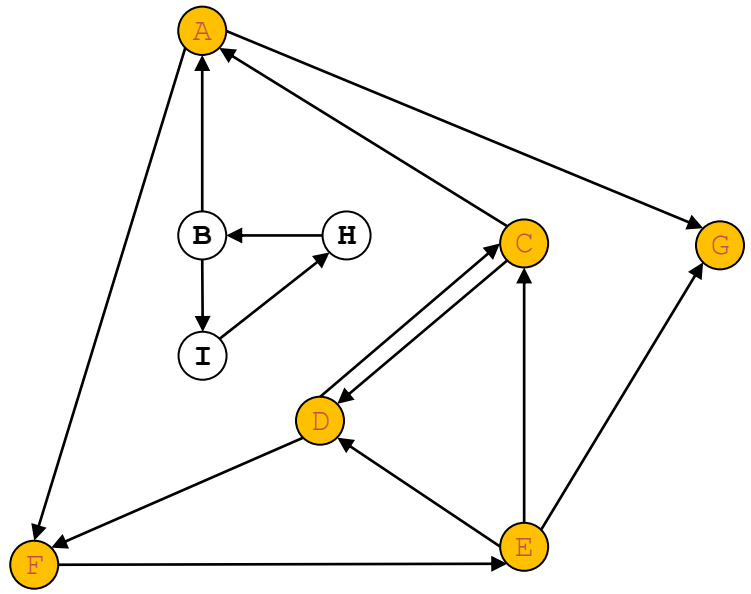
DFS 演示



系统栈:



DFS 演示



A结点可达的结点: A, C, D, E, F, G

B, I, H结点未访问, 以B为开始结点循环执行即可

若图的邻接关系为 $A \rightarrow B \rightarrow C, A \rightarrow D$ ，且邻接表按**字母顺序存储**（A 的邻接节点为 $[B, D]$ ），最终DFS的遍历顺序是（）

☒ A $A \rightarrow B \rightarrow C \rightarrow D$

☐ B $A \rightarrow D \rightarrow B \rightarrow C$

☐ C $A \rightarrow B \rightarrow D \rightarrow C$

☐ D $A \rightarrow D \rightarrow C \rightarrow B$

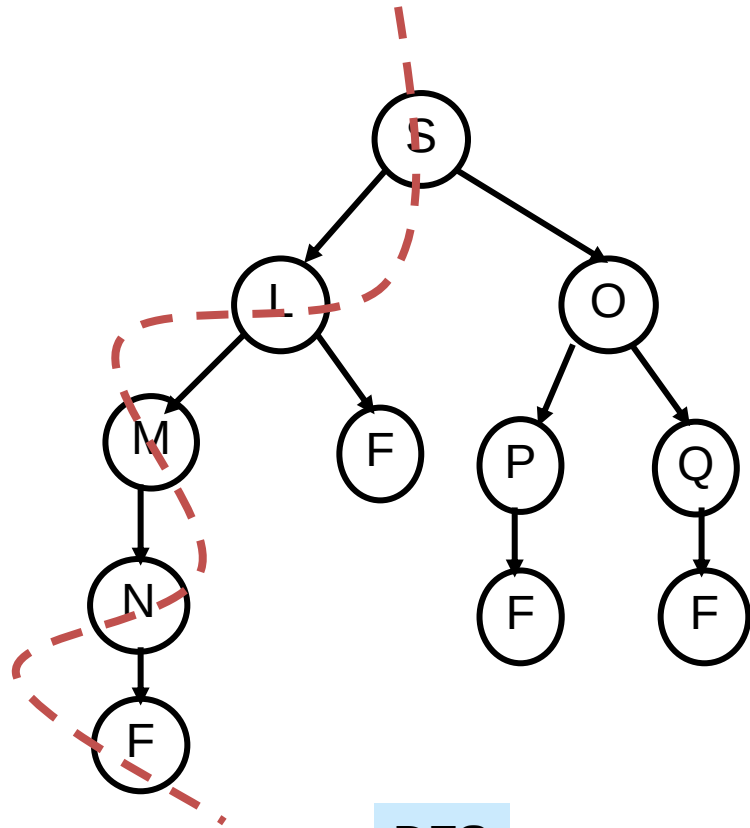
提交

DFS 练习:

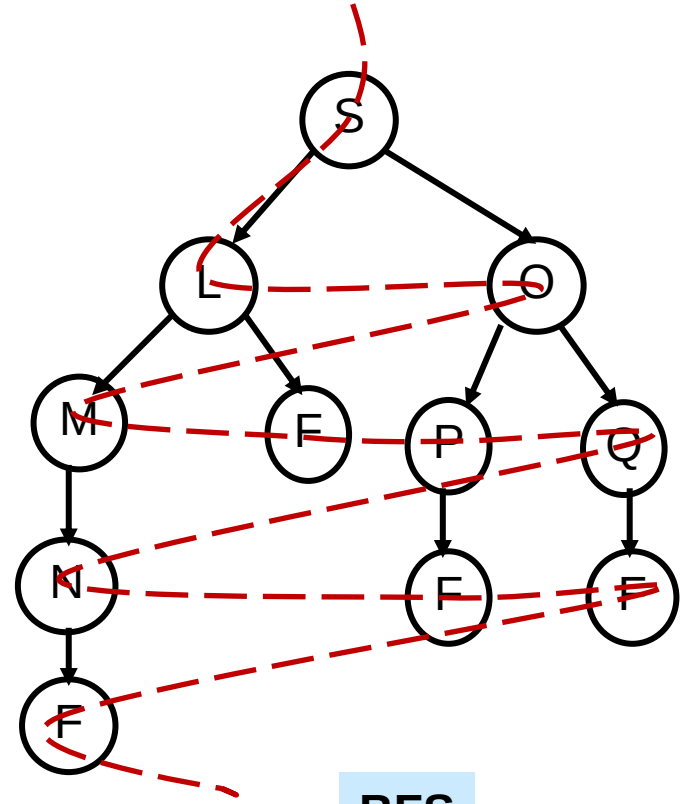
已知有向图的邻接矩阵如下，绘制出该有向图，并给出从顶点0出发的一种深度优先遍历序列。

$$\begin{bmatrix} 0 & 1 & 0 & 0 & 1 & 0 \\ 0 & 0 & 1 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

DFS & BFS



DFS



BFS

回溯算法

深度优先搜索是一种依照“**能进则进，不进则退**”的策略进行的**枚举算法**，也就意味着它可能**遍历整个状态空间图**，导致算法效率不高。给定一个问题的**状态空间**表示，设计搜索算法时需要考虑以下两个事实：

◆ 并不是所有的状态都是**合法状态**

◆ 并不是所有的状态都能达到**目标状态**

回溯算法 = 深度优先搜索 + 剪枝策略

回溯算法通常采用以下哪种策略遍历解空间？

- ☐ A 广度优先搜索 (BFS)
- ☒ B 深度优先搜索 (DFS)
- ☐ C 动态规划
- ☐ D 分治法

提交

回溯算法

回溯算法需要定义问题的**状态空间图**，包括以下四个方面：

1. **问题的状态表示**：一般用k元组表示，即 $X = [x_1, x_2, \dots, x_k]$ ，其中向量的维度，以及每一个分量的值域是状态表示的关键问题
2. **约束条件**：X中每一个**分量自身的取值约束；分量之间的取值约束**。注意：约束条件是剪枝策略的基础，需要深入分析和挖掘
3. **操作符集合（状态转移/决策分支）**：从一个状态转换到另外一个状态的操作，在状态空间图中它构成**边集**。
4. **问题解和解空间**：问题的解可以认为是一类**特殊的状态**，即**目标状态**。对于问题的一个实例，所有满足约束条件的解向量就构成该问题实例的解空间。

回溯算法

回溯算法需要设计合适的**剪枝策略**，尽量避免不必要的搜索。常用的剪枝策略包括两大类：

1. **约束函数剪枝**：根据约束条件，状态空间图中的部分状态可能是不合法的。因此，在状态空间图中**以不合法状态为根的子图/子树是不可能包含可行解**的，故其子空间不需要搜索。

- **约束函数剪枝** 可以剪除状态空间图中的**不可行解**

- **限界函数剪枝** 用于剪除状态空间图中的**可行但是非最优的解**

2. **限界函数剪枝**：这种策略一般应用于最优化问题。假设搜索算法当前访问的状态为 s ，且存在一个**判定函数**：它能判定**以 s 为根的子图/子树不可能包含最优解**，因此该子图/子树可以剪除而无需搜索

在回溯算法中，剪枝的主要目的是：

- ☐ A 减少递归深度
- ☐ B 优化空间复杂度
- ☒ C 提前终止无效分支的搜索
- ☐ D 合并重复子问题

提交

回溯算法 — 背包问题

【例-1】 对于 $n = 3$ 的0-1背包问题，其中背包容量 $c = 30$ ，每个物品的重量和价值分别为： $w = \langle 16, 15, 14 \rangle$, $p = \langle 45, 25, 35 \rangle$ 。

1) 问题的状态表示

$X = [x_1, x_2, \dots, x_k]$ 其中 k 表示已经处理过的物品数目， $x_i = \{0, 1\}$ ，如果 $x_i = 1$ ，则表示第 i 个物品装入背包，否则物品未装入。 $X = []$ 是初始状态，表示所有物品都没有处理。

2) 约束条件

- $x_i = \{0, 1\}$
- $\sum_{i=1}^k x_i w_i \leq C$

回溯算法 – 背包问题

【例-1】对于 $n = 3$ 的0-1背包问题，其中背包容量 $c = 30$ ，每个物品的重量和价值分别为： $w = \langle 16, 15, 14 \rangle$ ， $v = \langle 45, 25, 35 \rangle$ 。

3) 操作符

给定状态 $X = [x_1, x_2, \dots, x_k]$ ，可能的操作有两种：

- 把第 $k+1$ 个物品**装入**背包，有 $x_{k+1} = 1$ ；
- 把第 $k+1$ 个物品**不装入**背包，有 $x_{k+1} = 0$ ；

得到一个**新的状态** $X' = [x_1, x_2, \dots, x_k, x_{k+1}]$

4) 问题解和解空间

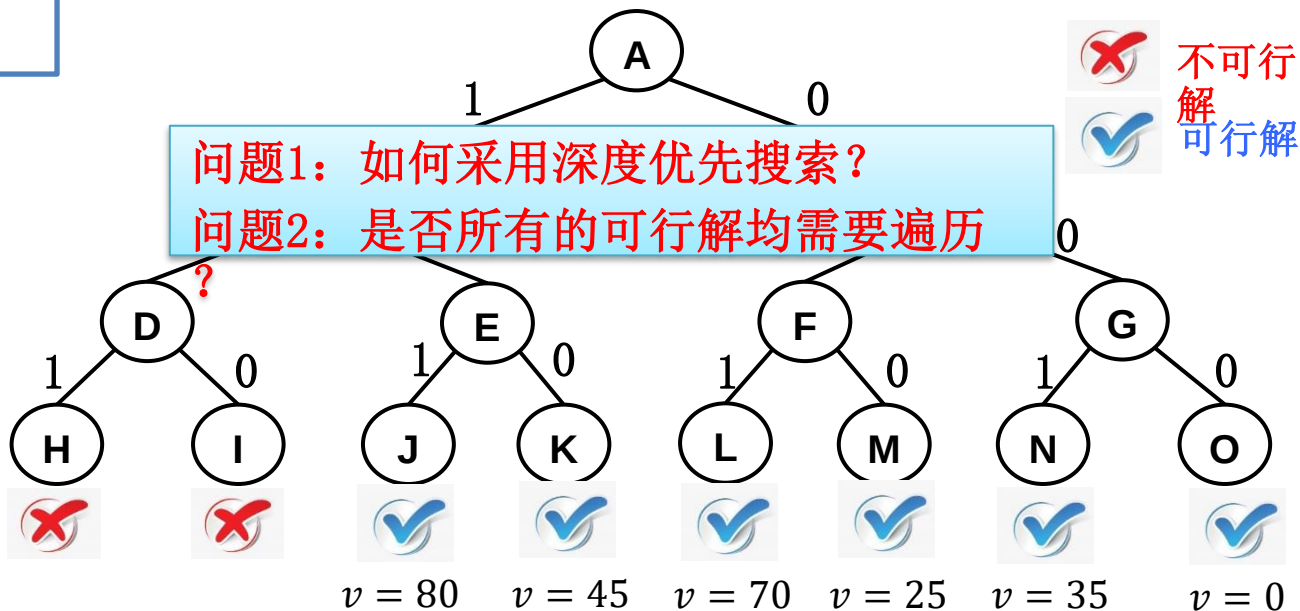
- 满足约束条件的任何**3维0-1向量** $[x_1, x_2, x_3]$ 都是0-1背包问题的一个解
- 解空间可以用一棵**二叉树**来表示，亦称为**子集树**

0-1背包问题

问题描述

$$n = 3, \quad C = 30, \quad w = \{16, 15, 14\}, \quad v = \{45, 25, 35\}$$

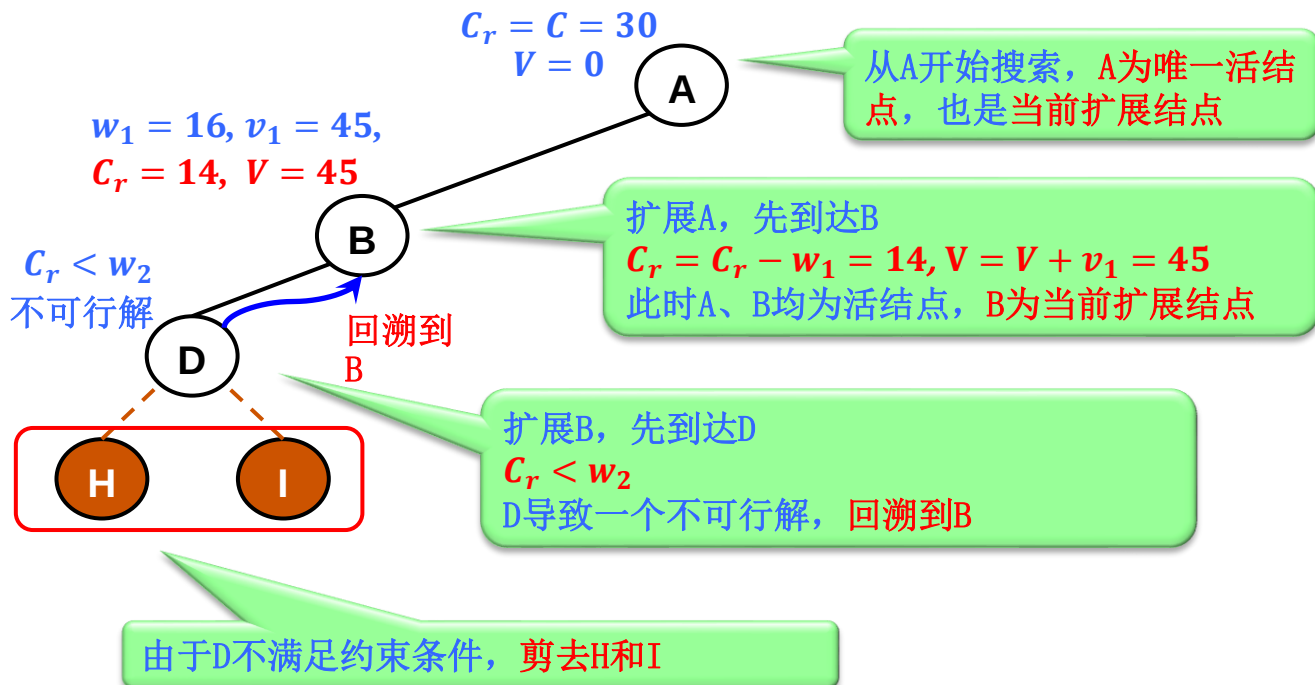
解空间



0-1背包问题

$n = 3$, $C = 30$, $w = \{16, 15, 14\}$, $v = \{45, 25, 35\}$

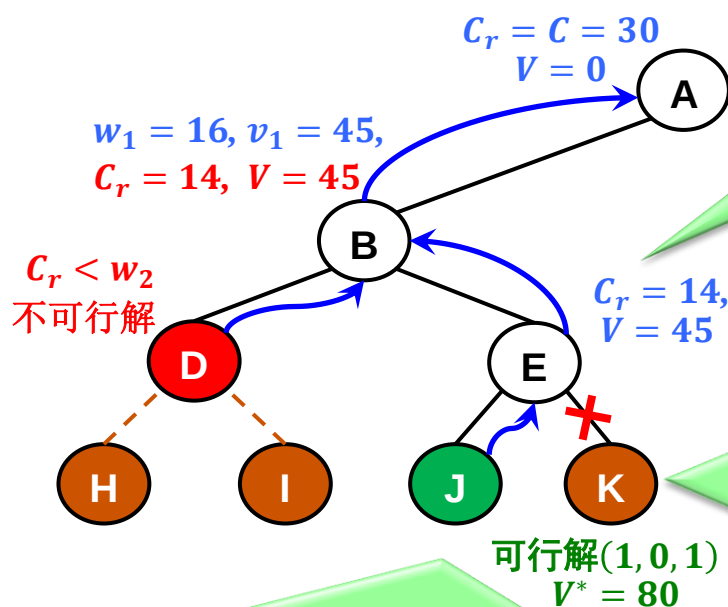
C_r : 当前背包容量, V : 当前背包价值



0-1背包问题

$n = 3$, $C = 30$, $w = \{16, 15, 14\}$, $v = \{45, 25, 35\}$

C_r : 当前背包容量, V : 当前背包价值



再扩展B到达E, E可行(物品2不选择)
A, B, E为活结点, E为新的扩展结点

判断以K为根的子树上界

(1) $V + V_K \leq V^*$, 搜索该子树无法提升价值, 剪枝。

(2) $V + V_K > V^*$, 扩展K。

K为叶结点且为子树根 $V_K = 0$,
且 $V + V_K = 45 \leq V^*$, 剪去K

E为死结点, 返回B

B为死结点, 返回A

扩展E, 先到达J, J可行

$C_r = C_r - w_3 = 0, V = V + v_3 = 45 + 35 = 80$ (当前最优值 $V^* = 80$)

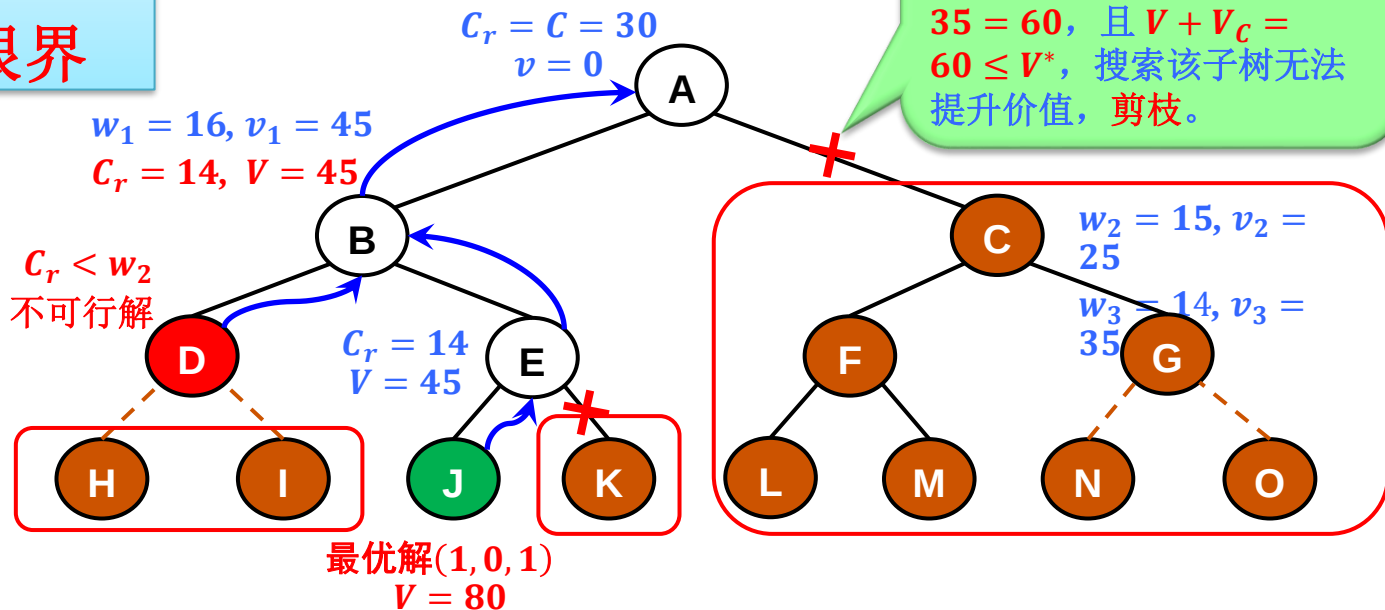
J不可扩展, 死结点, 获得一个可行解 (1, 0, 1), 返回到E

0-1背包问题

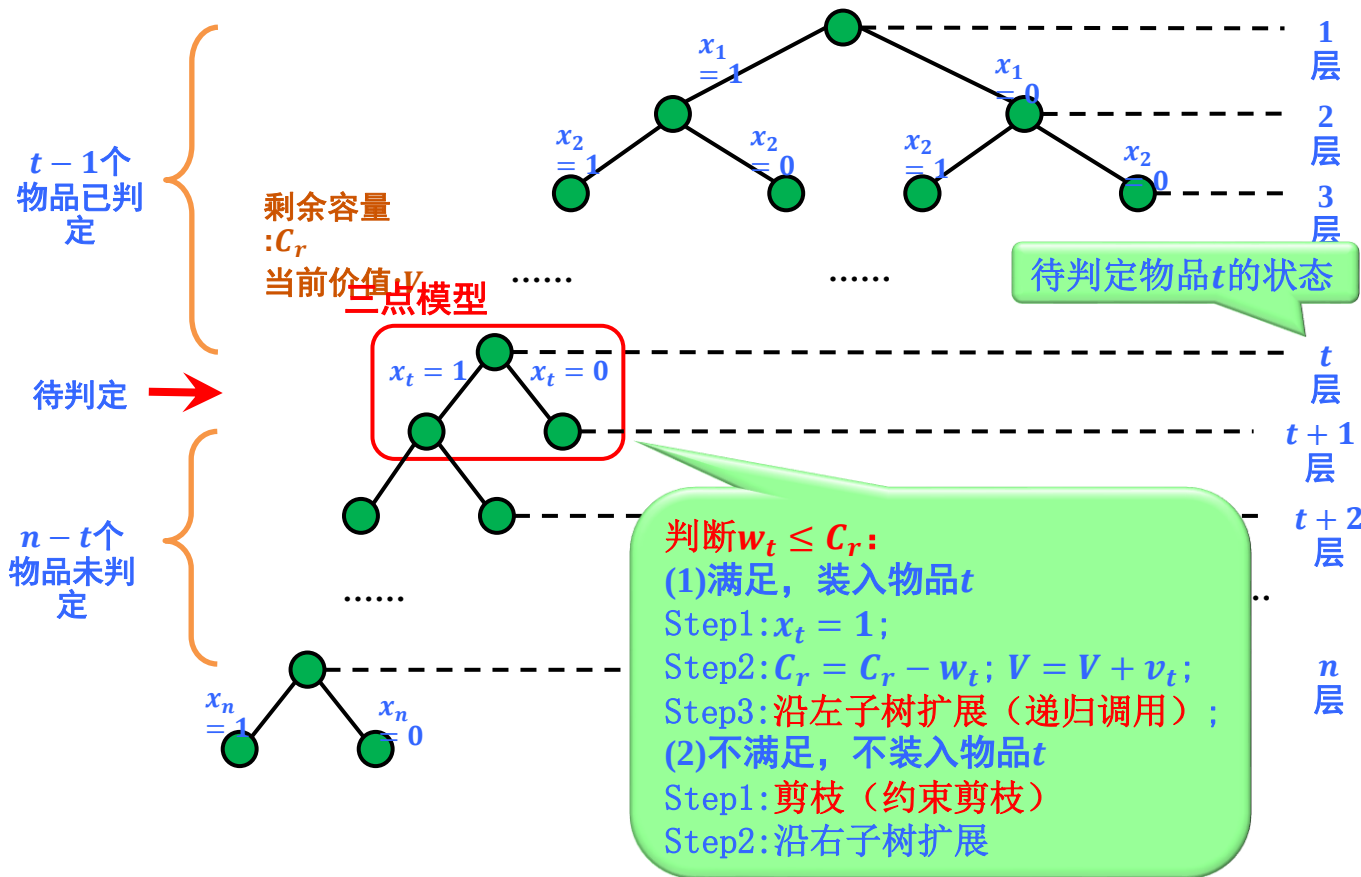
$n = 3$, $C = 30$, $w = \{16, 15, 14\}$, $v = \{45, 25, 35\}$

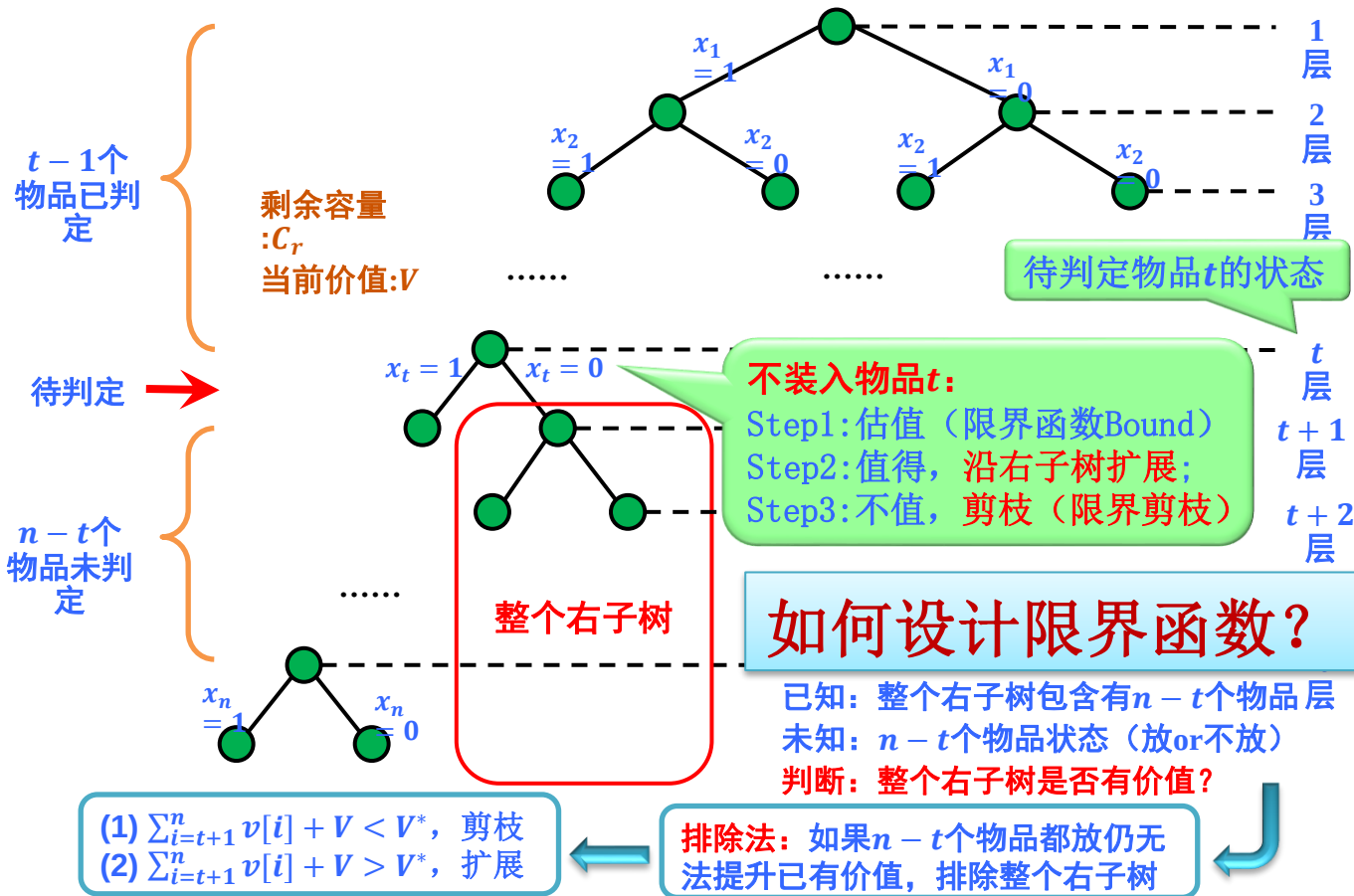
C_r : 当前背包容量, V : 当前背包价值

3次剪枝:
1次约束
2次限界



算法设计





深度遍历搜索：

```
void Backtrack(int t)
{
    if(t>n) // 已到叶节点，即所有物品均已判定
    { vBest = vPren; return; }
    if(w[t]<=cr) // 可放入物品t
    {
        x[t] = 1;
        cr = cr - w[t];
        vPren = vPren + v[t];
        Backtrack(t+1); // 扩展左子树
        cr = cr + w[t];
        vPren = vPren - v[t];
    }
    // 符合限界条件，扩展右子树
    if(Bound(t+1) + vPren > vBest)
        {x[t] = 0; Backtrack(t+1); }
}
```

放入物品t
更新cr及vPren

向上回溯
还原至父节点

限界函数（上界）：

```
double Bound(int t)
{
    vr = 0; // 剩余n-t个物品的价值和
    while(t<=n) {vr = vr+v[t]; t++;}
    return vr;
}
```

剩余物品总重量 > cr

剩余物品总价值，估值过高

塞满背包条件下，剩余物品总价值

缩小上界，加快剪枝

贪心策略

```
double Bound(int t)
{
    vr = 0; // 剩余n-t个物品的价值和
    while(t<=n && w[t]<=cr) // 放入可放入物品
    { cr = cr-w[t]; vr = vr+v[t]; t++; }
    if(t<=n) // 按单位价值放入不可放入物品
        vr = vr + v[t]/w[t]*cr;
    return vr;
}
```

回溯算法中，在左子树递归返回后执行`cr += w[t]`和`vPren -= v[t]`的目的是什么？

- ☐ A 保存当前最优解
- ☐ B 计算上界函数值
- ☒ C 恢复状态以保证右子树探索的独立性

提交

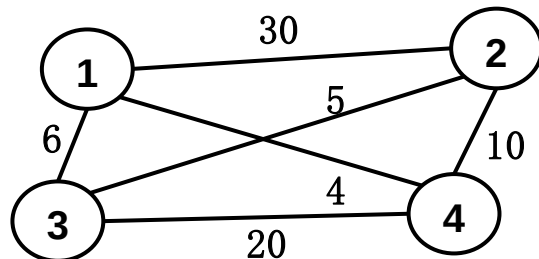
旅行售货员问题

问题描述

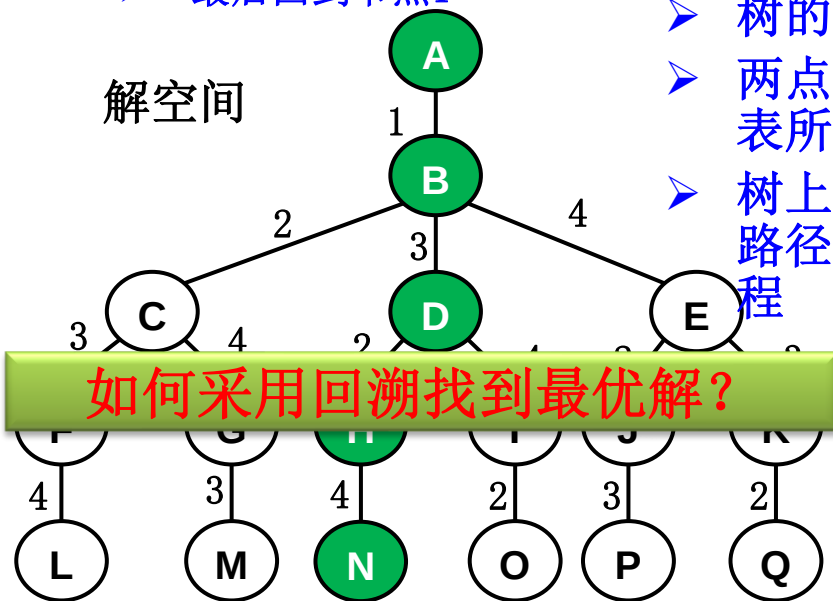
找到一条从节点1出发，经过每个城市一遍，最后回到节点1，使得**总路程最短路径**。

要求：

- 经过图中所有节点
- 出发点外各节点仅途径1次
- 最后回到节点1



解空间



如何采用回溯找到最优解？

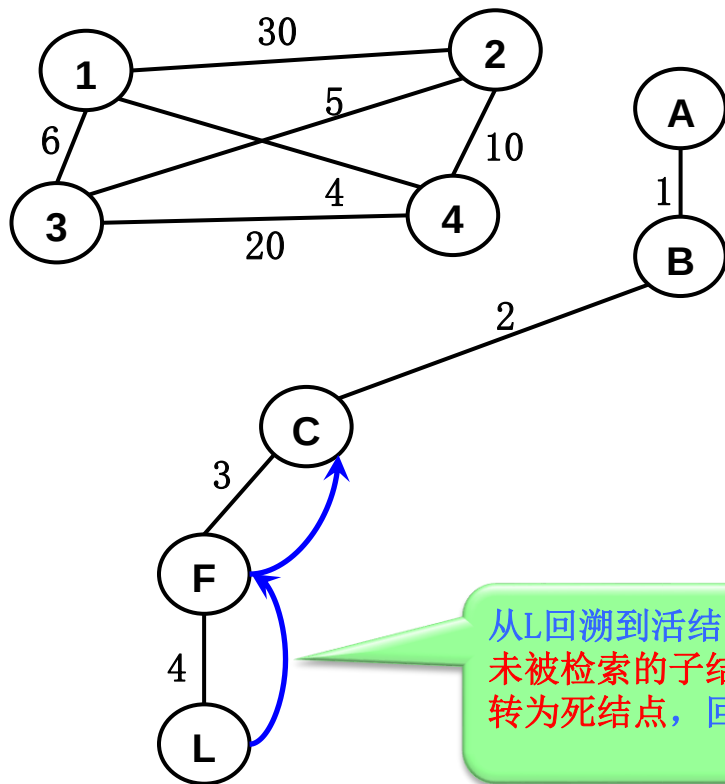
- 树的深度为5
- 两点之间路径上的标识数组代表所走城市
- 树上没有体现出回到城市1的路径，计算中默认包含此段路程

最优解：

1→3→2→4→1

最优值：25

旅行售货员问题 思路一：计算已走路线



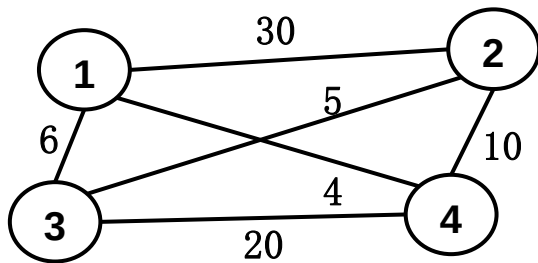
从A开始扩展，至叶结点L，
得到一个可行解
 $1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 1$ ，当前最优值
 $V^* = 30 + 5 + 20 + 4 = 59$

从L回溯到活结点F，F没有
未被检索的子结点，所以F
转为死结点，回溯到C

可行解： $1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 1$

最小值：59

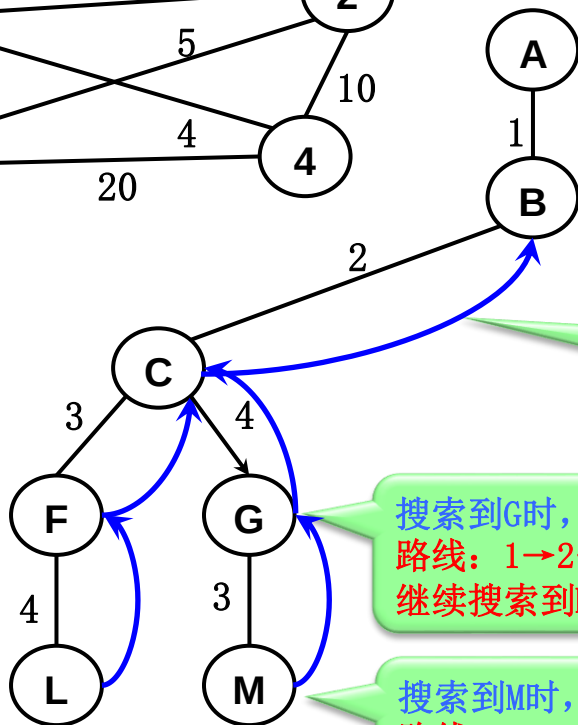
旅行售货员问题



深度搜索过程中，每到一个节点，
计算已走路线代价

(1) 代价 > 最优值，剪枝；

(2) 代价 < 最优值，继续搜索



C无未被检索的子结点，转
为死结点，回溯到活结点B

搜索到G时，判断A-G已走路线值

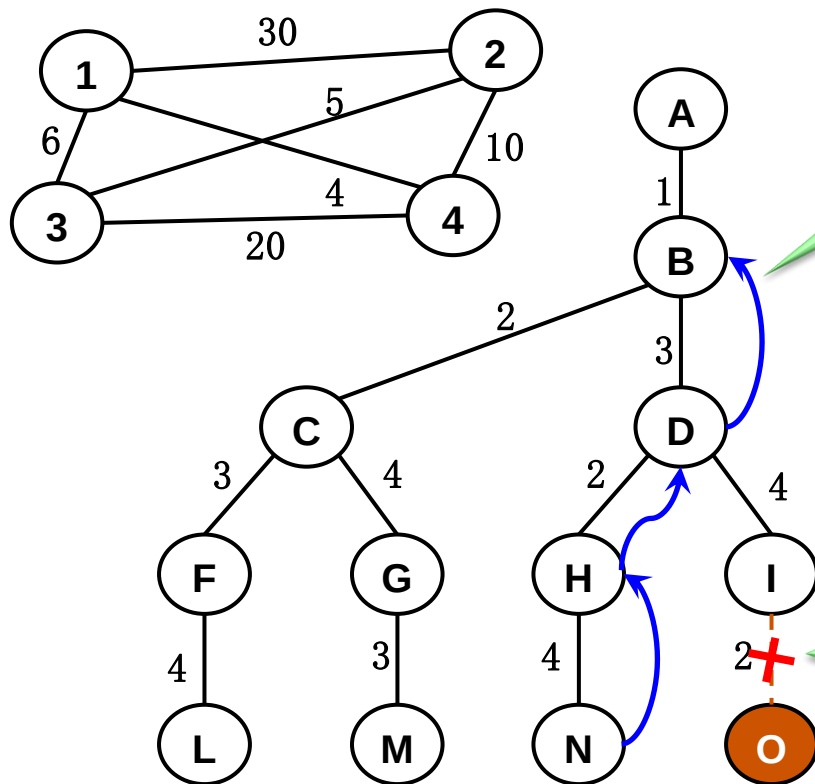
路线：1→2→4 ($V_G = 30 + 10 = 40 < V^* = 59$)，
继续搜索到M

搜索到M时，判断A-M已走路线值

路线：1→2→4→3→1 ($V_M = 30 + 10 + 20 + 6 = 66 > V^* = 59$)，非最佳路径，叶节点，回溯

可行解：1→2→3→4→1
最小值：59

旅行售货员问题



从结点N回溯到B

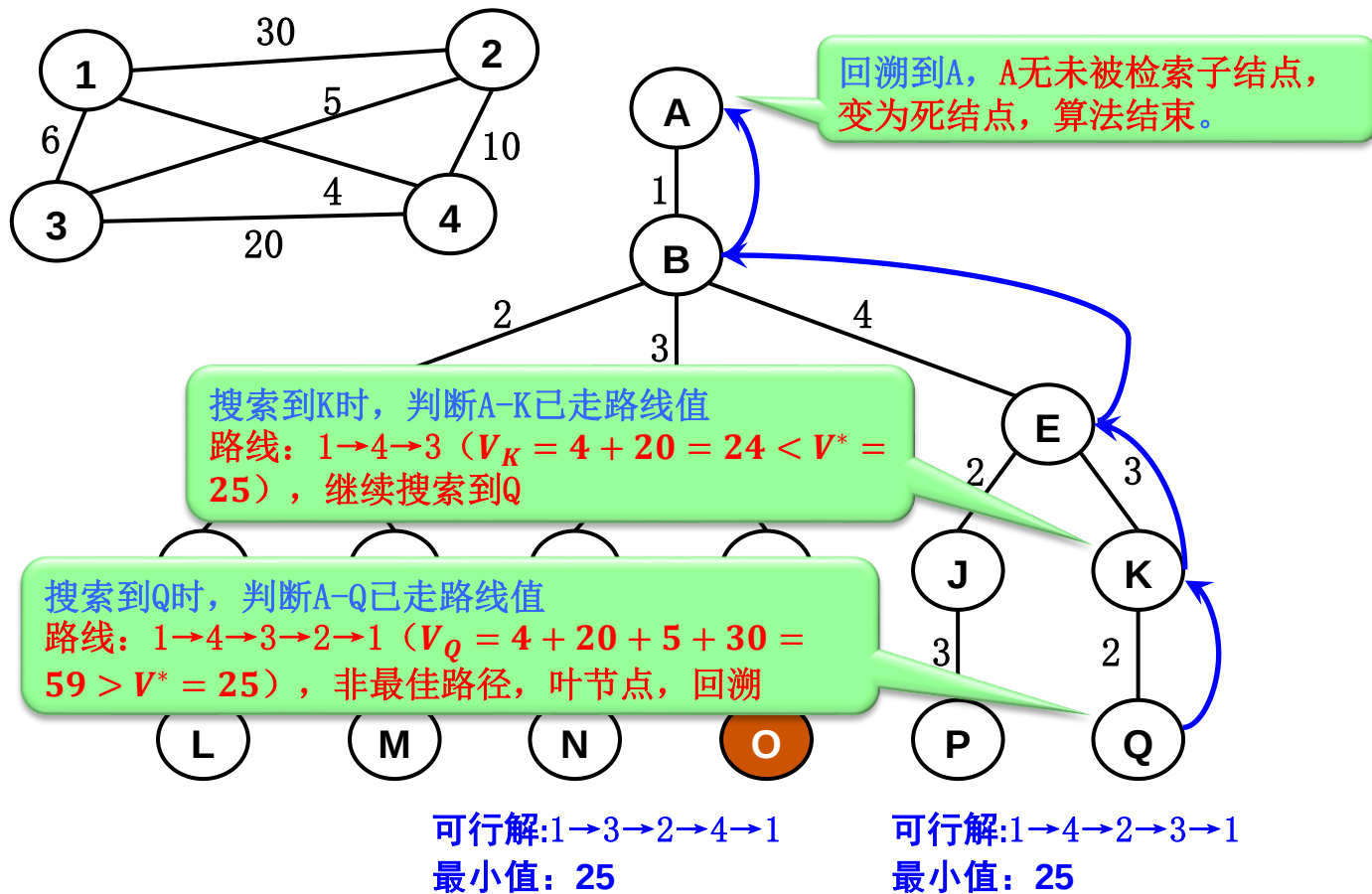
搜索到I时，判断A-I已走路
线值

路线：1→3→4 ($V_I = 6 + 20 = 26 > V^* = 25$)，已
无搜索意义，剪枝

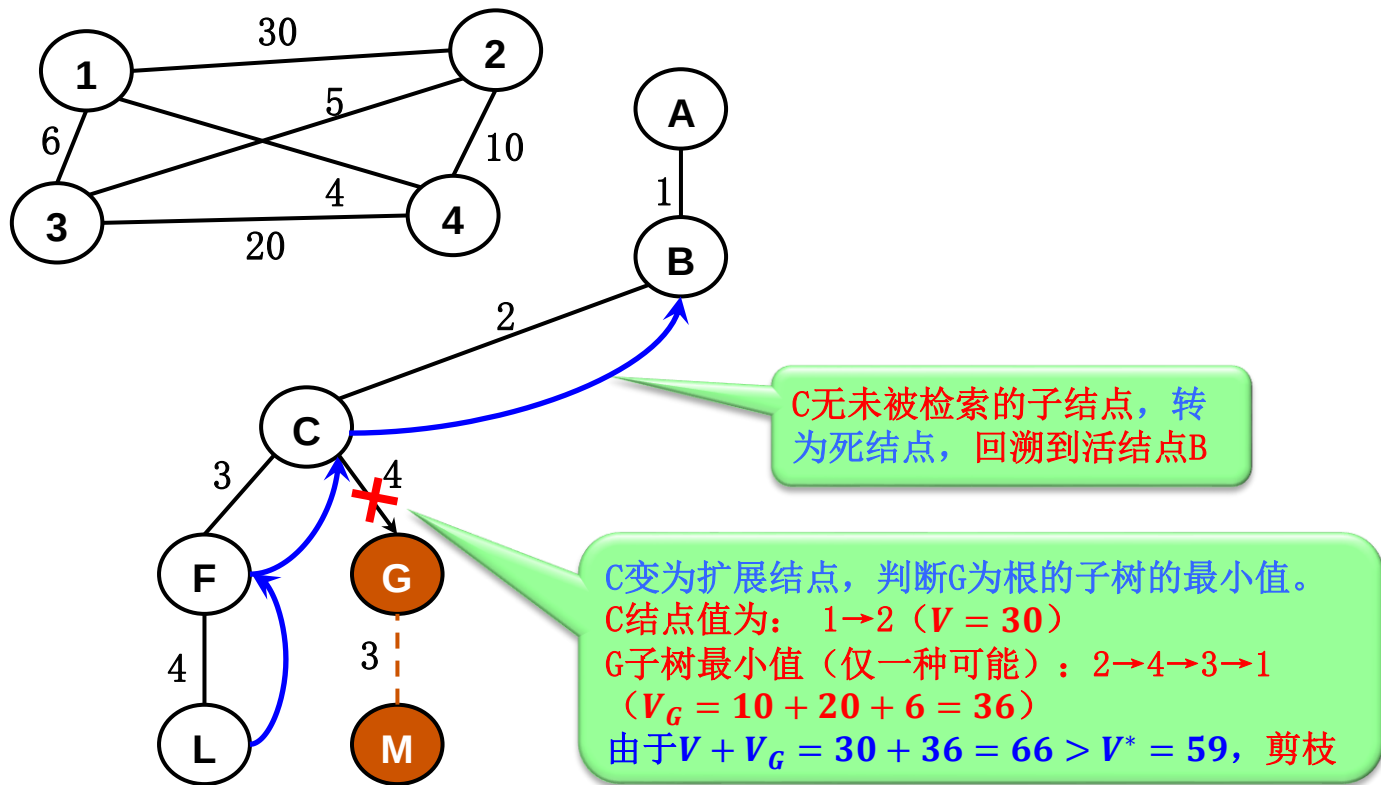
可行解：1→3→2→4→1

最小值：25

旅行售货员问题



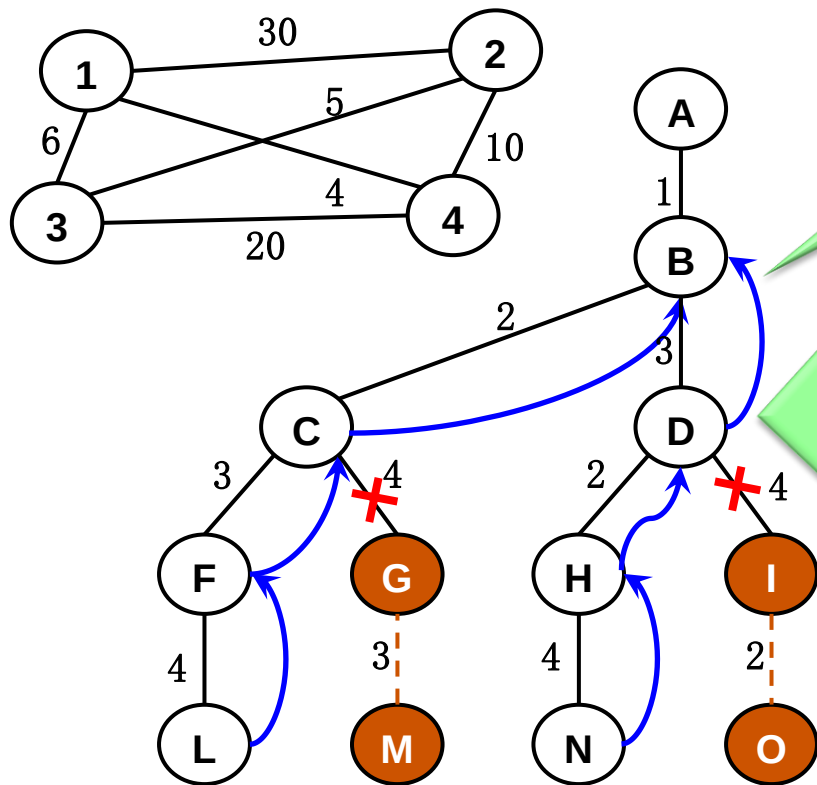
旅行售货员问题 思路二：估计未走路线



可行解: 1→2→3→4→1

最小值: 59

旅行售货员问题



从结点N回溯到B

B变为扩展结点，判断D为根的子树的最小值。

D结点值为：1→3 ($V = 6$)

D子树最小值 (2种可能)

➤ Case 1: 3→2→4→1 (选中)

$$V_D^1 = 5 + 10 + 4 = 19$$

➤ Case 2: 3→4→2→1 (剪枝)

$$V_D^2 = 20 + 10 + 30 = 60$$

Case 1下的最小值： $V +$

$$V_D^1 = 6 + 19 = 25 < V^* = 59$$

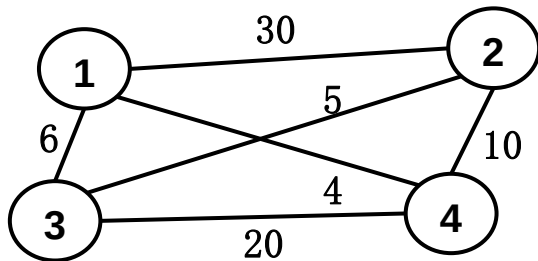
最佳解：1→3→2→4→1

最优值： $V^* = 25$ (更新)

可行解：1→3→2→4→1

最小值：25

旅行售货员问题



判断E为根的子树的最小值。

E结点值为: $1 \rightarrow 4$ ($V = 4$)

E子树最小值 (2种可能)

➤ Case 1: $4 \rightarrow 2 \rightarrow 3 \rightarrow 1$ (选中)

$$V_E^1 = 10 + 5 + 6 = 21$$

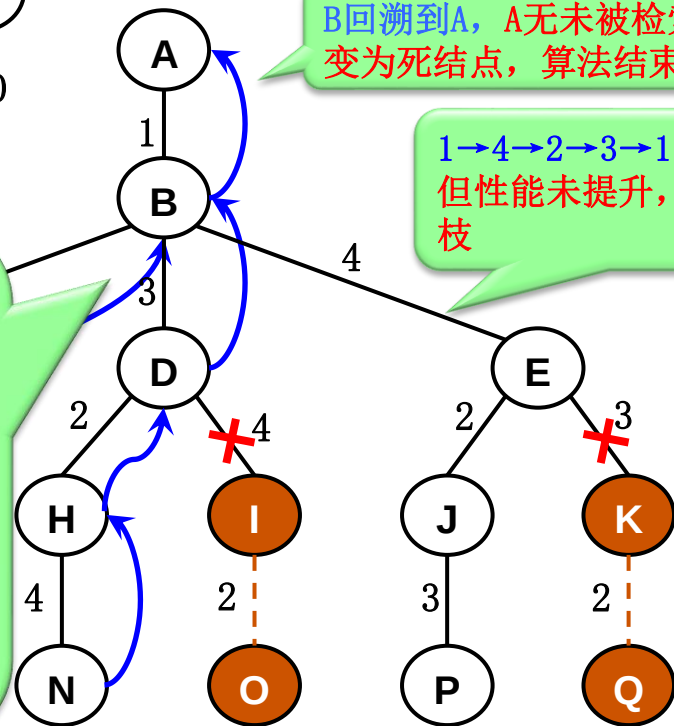
➤ Case 2: $4 \rightarrow 3 \rightarrow 2 \rightarrow 1$ (剪枝)

$$V_E^2 = 20 + 5 + 30 = 55$$

Case 1下的最小值: $V +$

$$V_E^1 = 4 + 21 = 25$$

与最佳解相同: $V^* = 25$



可行解: $1 \rightarrow 3 \rightarrow 2 \rightarrow 4 \rightarrow 1$
最小值: 25

可行解: $1 \rightarrow 4 \rightarrow 2 \rightarrow 3 \rightarrow 1$
最小值: 25

回溯算法实现

递归回溯

回溯法对解空间作深度优先搜索，因此，在一般情况下用递归方法实现回溯法， t 表示搜索深度。

```
void backtrack (int t)
{
    if (t>n) // t>n 表示算法已搜索到叶结点
        output(x); // 输出可行解x
    else
        for (int i=f(n,t);i<=g(n,t);i++)
            { // f(n,t)和g(n,t)为未搜索子树的起始和终止编号
                x[t]=h(i); // h(i)为当前扩展节点x[t]的第i个可选值
                if (constraint(t)&&bound(t))
                    backtrack(t+1); //递归调用
            }
}
```

constraint(t): 约束函数，剪去不可行解。
bound(t): 限界函数，剪去不可能最优的解。

回溯算法实现

迭代回溯

采用树的**非递归**深度优先遍历算法，可将回溯法表示为一个**非递归迭代过程**。

```
void iterativeBacktrack ()
{
    int t=1;
    while (t>0) {
        if (f(n,t)<=g(n,t))
            for (int i=f(n,t);i<=g(n,t);i++) {
                x[t]=h(i);
                if (constraint(t)&&bound(t)) {
                    if (solution(t)) output(x); // 输出最优解
                    else t++;} // 搜索下一层结点
                }
            else t--; // 回溯到上一个结点
    }
}
```

solution(t): 判断当前扩展
结点处是否已得到问题的
可行解

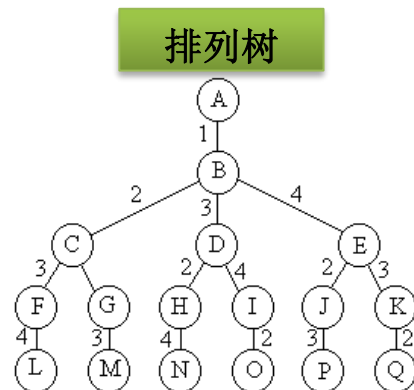
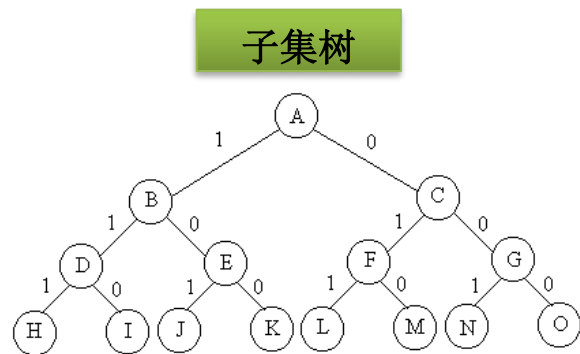
子集树与排列树

■ 子集树

- 从 n 个元素的集合 S 中找出满足某种性质的子集，相应的解空间称为子集树。通常有 2^n 个叶结点，结点总数为 $2^{n+1} - 1$ 。遍历时间： $O(2^n)$ 。
- 0-1背包问题的解空间为子集树。

■ 排列树

- 当所给问题的确定 n 个元素满足某种性质的排列时，相应的解空间树称为排列树。排列树通常有 $n!$ 个叶结点，遍历时间： $O(n!)$ 。
- 旅行售货员问题（TSP）的解空间为排列树。



装载问题

问题描述：有 n 个集装箱要装上2艘载重量分别为 C_1 和 C_2 的轮船，其中集装箱 i 的重量为 W_i ，且 $\sum W_i \leq C_1 + C_2$ 。请设计算法确定是否有一个合理的装载方案可将这些集装箱上这2艘轮船。

输入：多组测试数据。每组测试数据包括两行：第一行输入集装箱数目 n ($n < 1000$)，以及两艘轮船的载重 C_1 和 C_2 ；第二行输入 n 个整数，表示每个集装箱的重量。

输出：如果存在合理装载方案，输出第一艘轮船的最大装载重量；否则，输出“No”。

输入样例：

```
3  50 50
10 40 40
3  50 50
20 40 40
```

输出样例

```
50
No
```

把原问题简化为一艘轮船的装载问题。容易证明下述结论：如果一个给定装载问题实例有解，则采用下面的策略可得到最优装载方案：

- (1)首先将第一艘轮船**尽可能装满**；
- (2)将剩余的集装箱装上第二艘轮船。

问题分析

装载问题等价于以下特殊的0-1背包问题：

$$\begin{aligned} \max \quad & \sum_{i=1}^n w_i x_i \\ \text{st.} \quad & \begin{cases} \sum_{i=1}^n w_i x_i \leq C_1 \\ x_i \in \{0,1\}, 1 \leq i \leq n \end{cases} \end{aligned}$$

问题分析

1. 问题的状态表示

$$X = [x_1, x_2, \dots, x_k]$$

2. 约束条件

$$\sum_{i=1}^k x_i w_i \leq C_1, x_i \in \{0, 1\}$$

3. 操作符

给定状态 $X = [x_1, x_2, \dots, x_k]$, 可能的操作有两种:

① 把第 $k+1$ 个集装箱装上轮船, 有 $x_{k+1} = 1$

② 把第 $k+1$ 个集装箱不装轮船, 有 $x_{k+1} = 0$

得到新的状态 $X' = [x_1, x_2, \dots, x_k, x_{k+1}]$

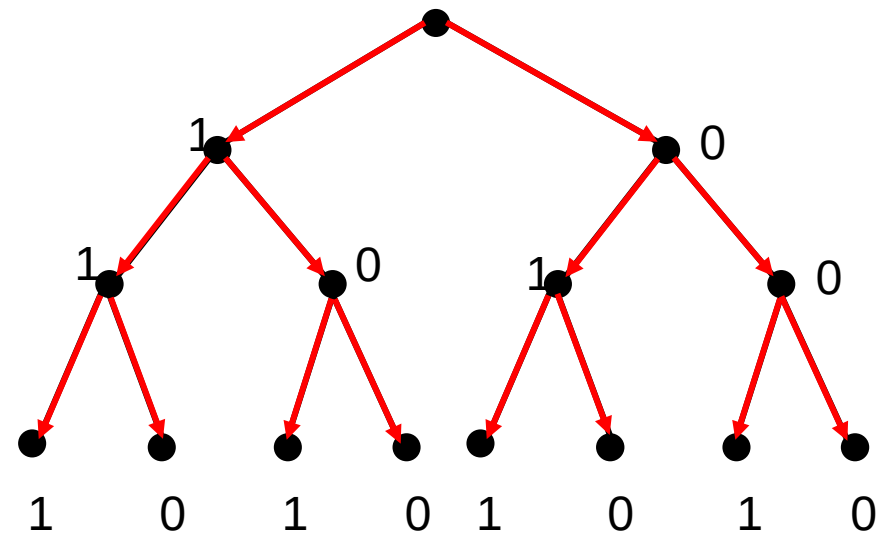
4. 问题解和解空间

满足约束条件的任何 n 维0-1向量 $[x_1, x_2, \dots, x_n]$ 都是一个可行解, 解空间可以用一棵二叉树来表示

深度优先搜索解法

```
#define MaxBox 1000
int globalWeight[MaxBox],globalNum,globalC1; //输入参数
int globalX[MaxBox],globalAns; //保存状态X和最优值的全局变量
void loadingDFS(int t) {
    if(t == globalNum) { //边界条件, 判定一个n维-1向量是否是可行解
        int sumWeight1=0;
        for(int i=0; i<globalNum; i++) {
            sumWeight1 += globalX[i]*globalWeight[i];
        } //计算装载量
        if((sumWeight1<=globalC1) && sumWeight1>globalAns)
            globalAns = sumWeight1; //找到更优的可行解
        return;
    } // end of if
    globalX[t]=1;
    loadingDFS(t+1); //扩展左子树
    globalX[t]=0;
    loadingDFS(t+1); //扩展右子树
}
```

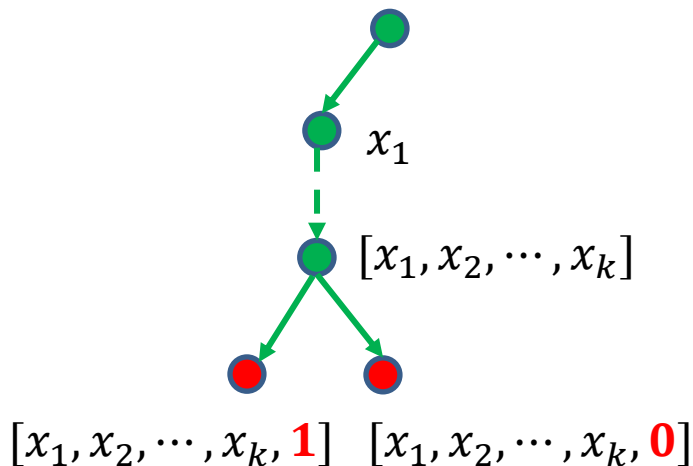
深度优先搜索演示



8个状态全部访问一遍，选择最大且小于C1的结果输出

回溯算法

1) **约束函数剪枝** 用于剪除那些**不可能导出可行解**的分支。给定任意状态 $[x_1, x_2, \dots, x_k]$, 怎么判断其子结点是否可行?



■ 左子结点

$$\sum_{i=1}^{k+1} w_i x_i = \sum_{i=1}^k w_i x_i + w_{k+1}$$

$$Wt = \sum_{i=1}^k w_i x_i$$

$$Wt + w_{k+1} > C_1$$

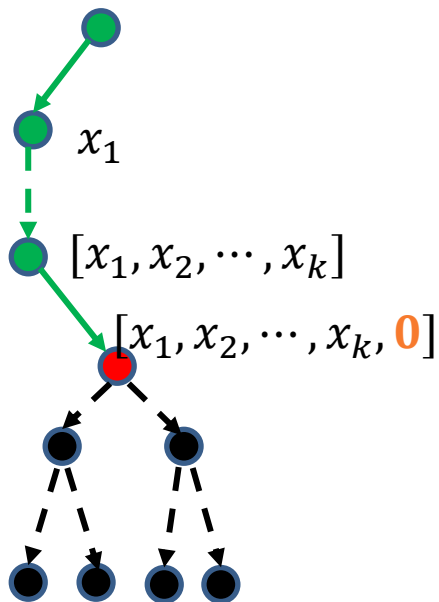
■ 右子结点

$$Wt \leq C_1, x_{k+1} = 0$$

回溯算法

2) 限界函数剪枝 用于剪除那些不可能导出最优解的分支。给定任意状态 $[x_1, x_2, \dots, x_k]$ ，怎么判断其子结点是否可能包含最优解？

$$Wt(X) + Bd(X') \leq MaxWt$$



■ 右子结点

假设 $X_s = [X, X']$ 为包含 X 的解向量，

其中 $X = [x_1, x_2, \dots, x_k, 0]$, $X' = [x_{k+2}, \dots, x_n]$

$$\begin{aligned} Wt(X_s) &= \sum_{i=1}^{k+1} w_i x_i + \sum_{i=k+2}^n w_i x_i \\ &= Wt(X) + Wt(X') \end{aligned}$$

$Wt(X')$ 未知!

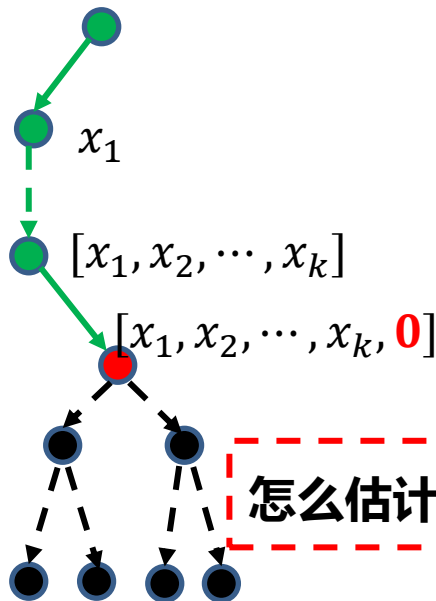
➤ 估计 $Wt(X')$ 的上限 $Bd(X')$ ，即有 $Bd(X') \geq Wt(X')$

➤ 记录当前已得到的最优解，记为 $MaxWt$

回溯算法

2) **限界函数剪枝** 用于剪除那些**不可能导出最优解**的分支。给定任意状态 $[x_1, x_2, \dots, x_k]$, 怎么判断其子结点是否可能包含最优解?

$$Wt(X) + Bd(X') \leq MaxWt$$



■ 右子结点

$$\triangleright Bd(X') = (n - k - 1) * maxW$$

$$\triangleright Bd(X') = \sum_{i=k+2}^n w_i$$

■ 左子结点

$$[x_1, x_2, \dots, x_k, 1]$$

装载重量的上限等于其父结点的上限, **不需判定!**

回溯算法

回溯

```
#define MaxBox 1000
int globalWeight[MaxBox],globalNum,globalC1;
int globalX[MaxBox];
int globalWt,globalBd,globalMaxWt;
void loadingBacktrack(int t) {
    if(t == globalNum) {
        globalMaxWt = globalWt;
        return;
    } // end of (if
    globalBd -= globalWeight[t];
    if(globalWt + globalWeight[t] <= globalC1) {
        globalX[t] = 1;
        globalWt += globalWeight[t];
        loadingBacktrack(t+1);
        globalWt -= globalWeight[t];
    }
    if(globalWt + globalBd > globalMaxWt) {
        globalX[t] = 0;
        loadingBacktrack(t+1);
    }
    globalBd += globalWeight[t];
}
```

//输入参数
//保存状态X的全局变量
//保存中间量的全局变量
//边界条件，得到一个C1的更好可行解
//扩展子结点时减少globalBd
//约束剪枝
//增大globalWt
//扩展左子树
//回溯时恢复globalWt
//限界剪枝
//扩展右子树
//回溯时恢复globalBd

回溯算法演示

剪枝策略:

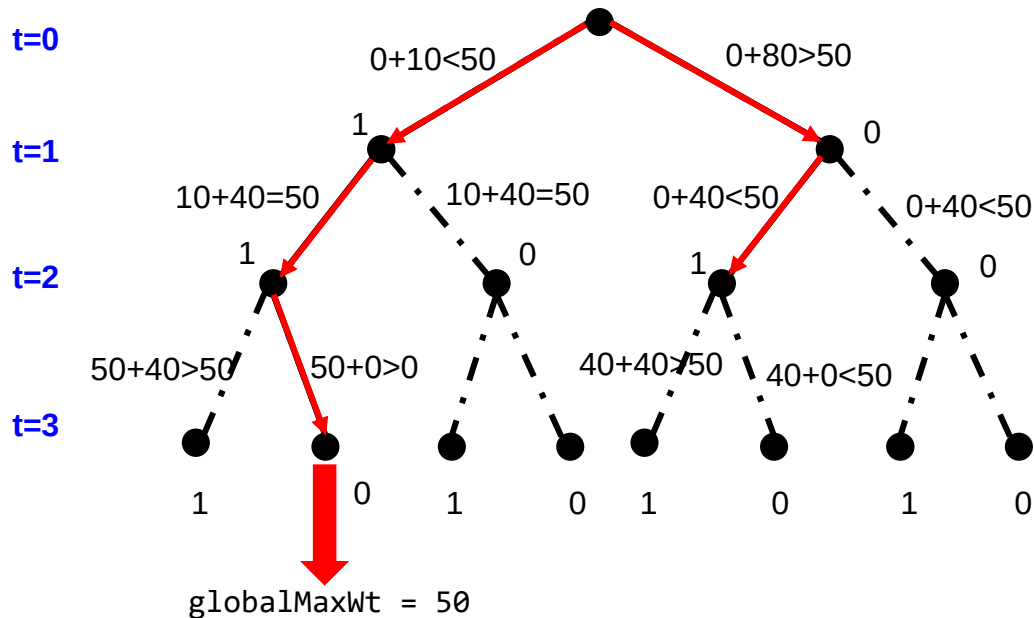
左子树: $\text{globalWt} + \text{globalWeight}[t] \leq \text{globalC1}$

右子树: $\text{globalWt} + \text{globalBd} > \text{globalMaxWt}$

输入样例:

3 50 50

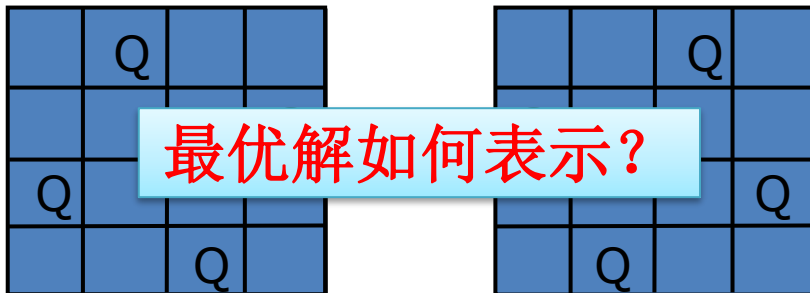
10 40 40



N后问题

问题描述：在 $n \times n$ 格的棋盘上放置彼此不受攻击的 n 个皇后，使他们中任意2个之间无相互“攻击”，即**任何2个皇后不放在同一行或同一列或同一斜线上**。

四后问题的解



最优解如何表示?

算法思路

■ **解空间：** (x_1, x_2, x_3, x_4) , x_i 为皇后 i 放在棋盘第 i 行的第 x_i 列

■ **约束条件：**

(1) **显式约束：** 皇后 i, j 不在**同一列**上，即 $x_i \neq x_j$

(2) **隐式约束：** 皇后 i, j 不在**同一斜线**上，即
$$\begin{cases} x_i - i \neq x_j - j \\ x_i + i \neq x_j + j \end{cases}$$
 等价于 $|i - j| \neq x_i - x_j$

N后问题

问题描述： 在 $n \times n$ 格的棋盘上放置彼此不受攻击的 n 个皇后，
即任意2个皇后不可放在同一行、同一列或同一斜线上

四后问题的解

(x_1, x_2, x_3, x_4)
 $= (2, 4, 1, 3)$

	Q		
			Q
Q			
		Q	

(x_1, x_2, x_3, x_4)
 $= (3, 1, 4, 2)$

		Q	
Q			
			Q
	Q		

算法思路

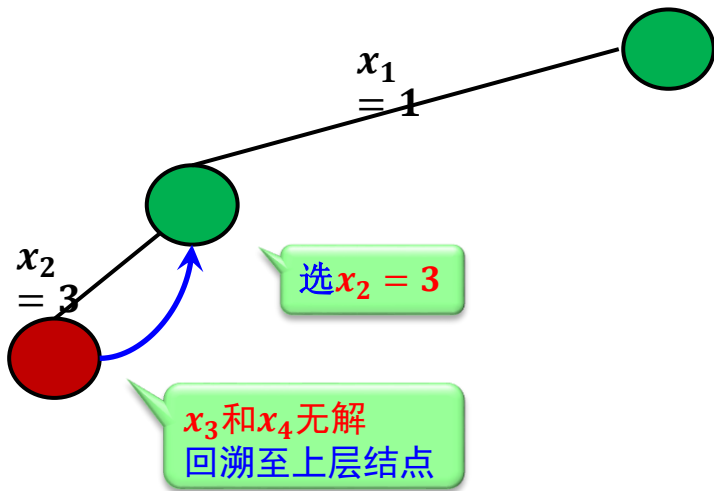
■ **解空间：** (x_1, x_2, x_3, x_4) ， x_i 表示皇后 i 放在棋盘的 (i, x_i) 单元

■ **约束条件：**

(1) **显式约束：** 皇后 i ， j 不在**同一列**上，即 $x_i \neq x_j$

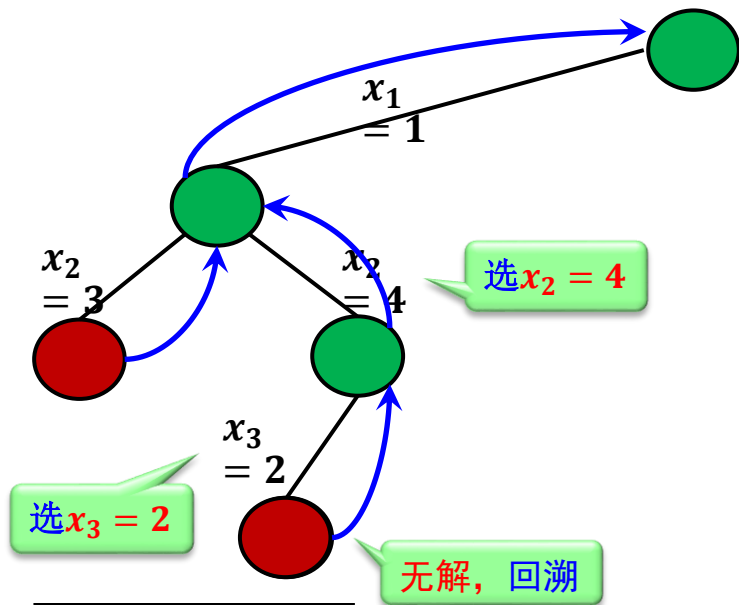
(2) **隐式约束：** 皇后 i ， j 不在**同一斜线**上，即
$$\begin{cases} x_i - i \neq x_j - j \\ x_i + i \neq x_j + j \end{cases}$$
等价于 $|i - j| \neq x_i - x_j$

N后问题



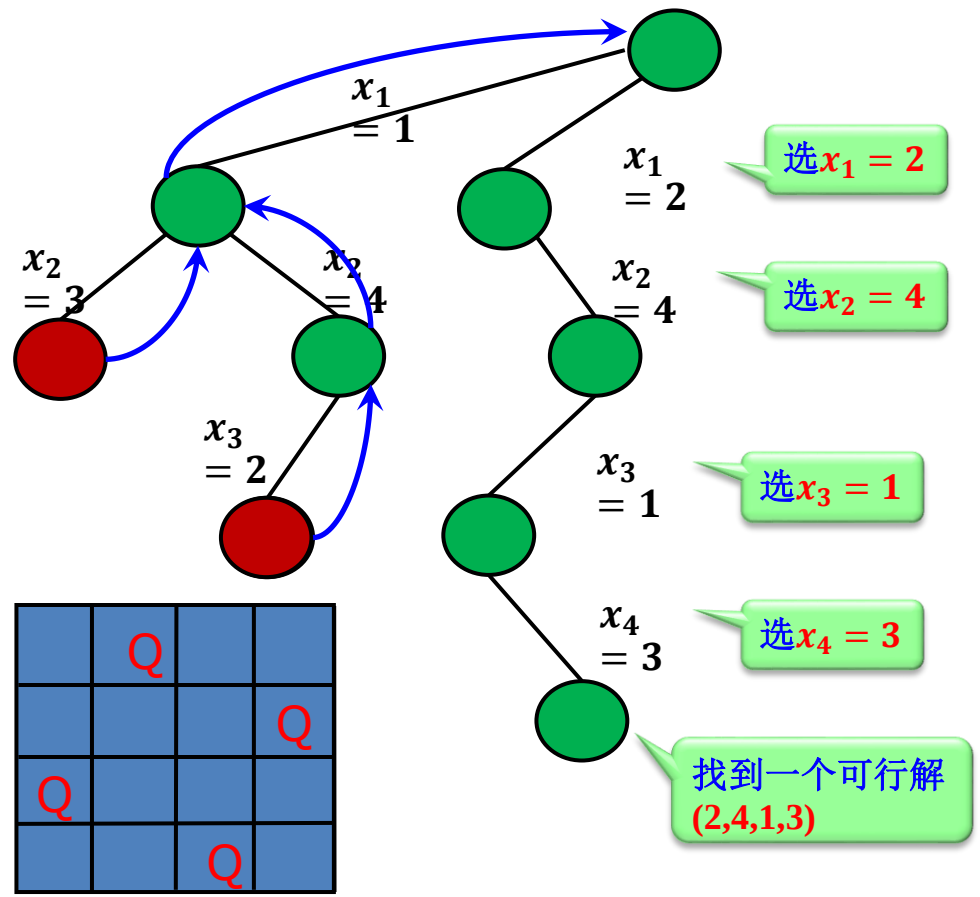
Q			
		Q	

N后问题

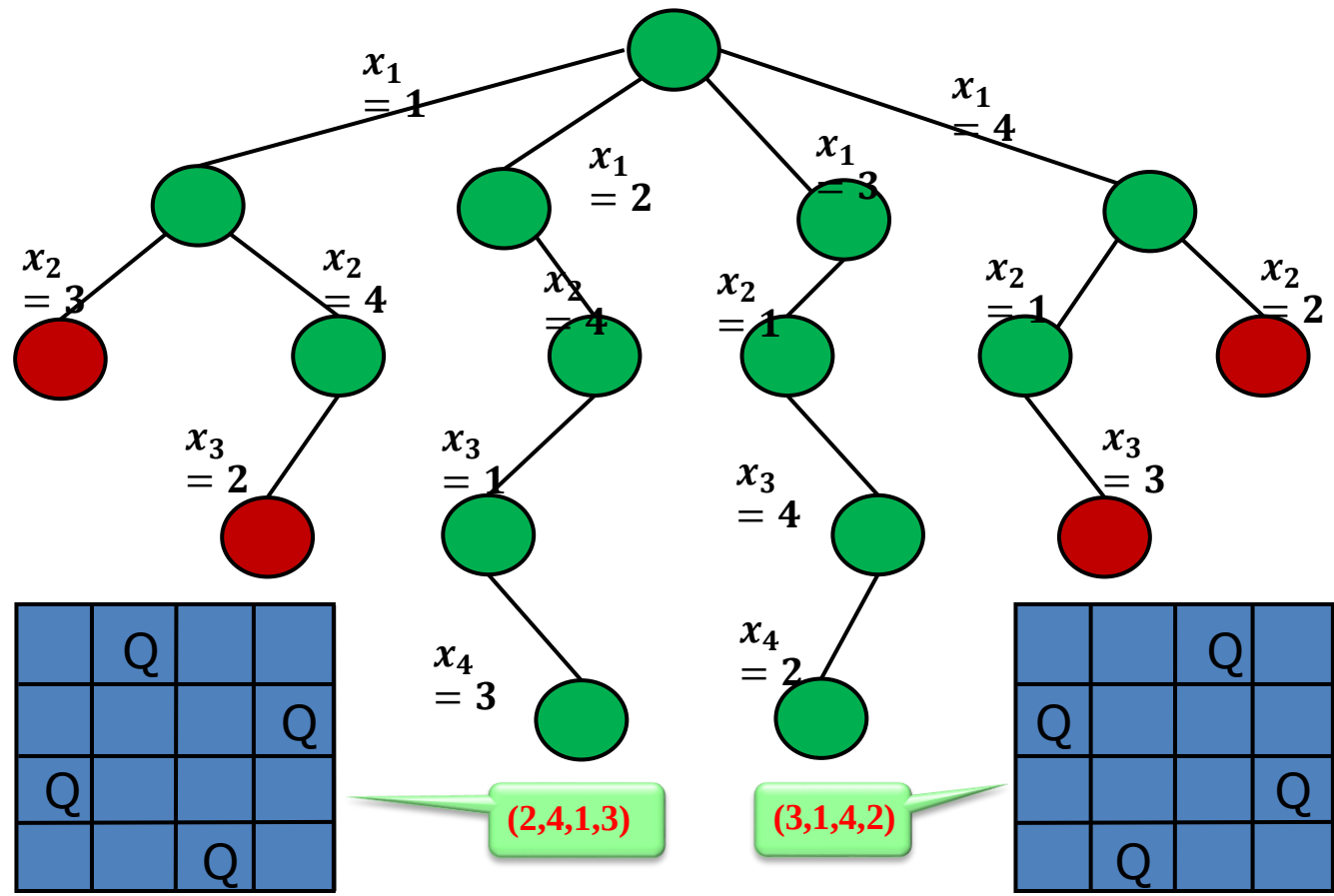


Q			
			Q
	Q		

N后问题



N后问题



```

#define N 4
int queens[N]; // 存储皇后位置的数组
int solution_count = 0; // 解的计数
// 检查皇后是否安全放置在(row, col)位置
bool is_safe(int row, int col)
{
    for (int i = 0; i < row; i++)
    {
        if (queens[i] == col || // 检查列
            queens[i] - i == col - row || // 检查主对角线
            queens[i] + i == col + row) // 检查副对角线
        {
            return false;
        }
    }
    return true;
}

```

```

{
    if (col >= N) { // 所有皇后都已成功放置
        solution_count++;
        printf("Solution #d:\n", solution_count);
        for (int i = 0; i < N; i++) {
            for (int j = 0; j < N; j++) {
                if (j == queens[i])
                {
                    printf("Q ");
                }
                else
                {
                    printf(". ");
                }
            }
            printf("\n");
        }
        printf("\n");
        return true;
    }

    for (int i = 0; i < N; i++) {
        if (is_safe(col, i)) {
            queens[col] = i;
            solve_n_queens(col + 1);
            // 回溯
            queens[col] = 0;
        }
    }
    return false;
}

int main() {
    solve_n_queens(0);
    printf("Total solutions found: %d\n", solution_count);
    return 0;
}

```


Solution #1:

```
. Q . .  
. . . Q  
Q . . .  
. . Q .
```

Solution #2:

```
. . Q .  
Q . . .  
. . . Q  
. Q . .
```

Total solutions found: 2

Solution #88:
. . . . Q .
. . . . Q .
. . Q . . .
Q
. . . . Q .
Q
. . . Q . .

Solution #89:
. Q
Q
. . . Q . . .
Q
. . . . Q .
. . . Q . . .
. . . . Q .

Solution #90:
. Q
Q
. . . Q . . .
Q
. . . . Q .
. . . . Q .
. . . . Q .

Solution #91:
. Q
. . Q
Q
. . . . Q .
Q
. . . Q . . .
. . . . Q .

Solution #92:
. Q
. . . Q . . .
Q
. . Q
. . . . Q .
Q
. . . . Q .
. . . . Q .

Total solutions found: 92

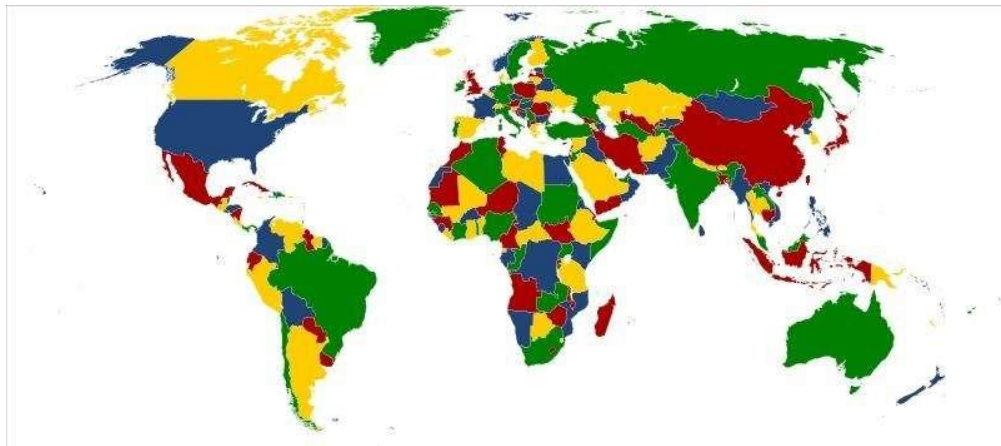
Process exited after 1.316 seconds with return value 0 (265.6 ms cpu time, 3292 KB mem used).

图的 m 着色问题

图的 m 可着色判定问题

给定无向连通图 G 和 m 种不同的颜色，用 m 种颜色为图 G 的各顶点着色，每个顶点着一种颜色，是否有一种着色法使 G 中每条边的2个顶点着不同颜色？

- 图的色数 m ：最少需要的颜色数量
- 可着色优化问题：求图色数 m 的问题



图的m着色问题

四色定理

任何一张平面地图只用四种颜色就能使具有共同边界的国家着上不同的颜色。

世界近代三大数学难题之一

（费马大定理，四色定理，哥德巴赫猜想）

背景

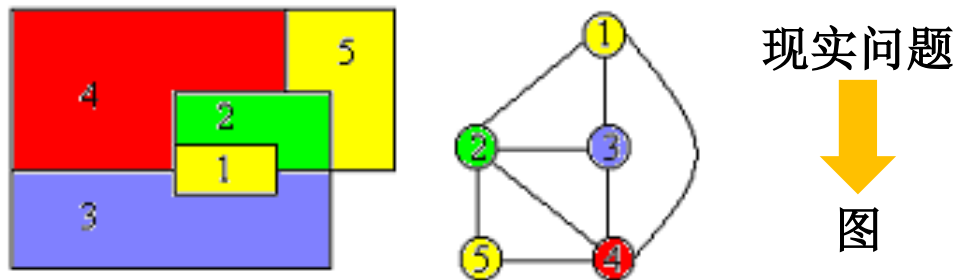
1852（四色猜想）：伦敦制图员发现任意地图仅用4中颜色即可区分所有区域

1872：英国数学家凯利向伦敦学会提出四色猜想，由此登上世界数学舞台；

1878-1880（五色定理）：数学家肯普和泰勒提交了四色猜想的证明论文，但实际证明的是五色定理；

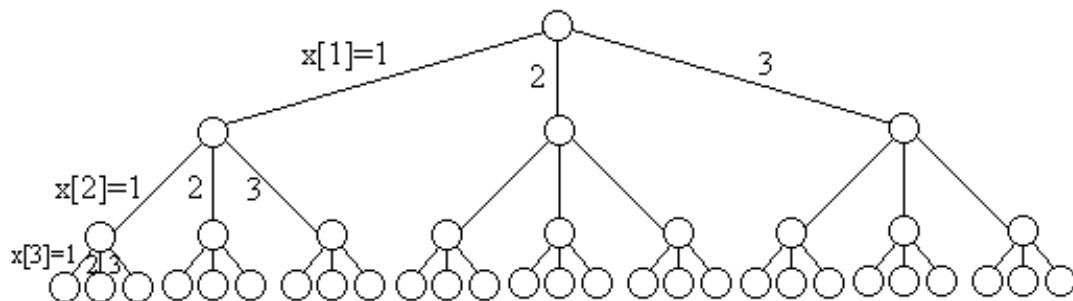
1976（四色定理）：美国数学家阿佩尔和德国数学家哈肯利用计算机构造了1万个图形，并对其中2千张特殊图形，经过1200小时2百亿次逻辑判定，证明了在1900多个着色区域内没有任何一种图形需要5种及以上颜色。由于1900多种已远远高于人力从逻辑上证明的区域数量，故认为四色定理被证明。

图的m着色问题

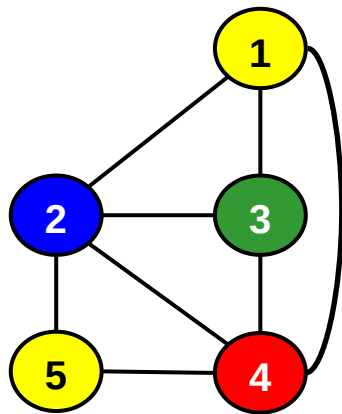


- **解空间:** $(x_1, x_2, \dots, x_n)$ 表示顶点 i 着颜色 x_i
- **约束条件:** 顶点 i 与已着色的相邻顶点颜色不重复

$n = 3, m = 3$ 时的解空间树



图的m着色问题

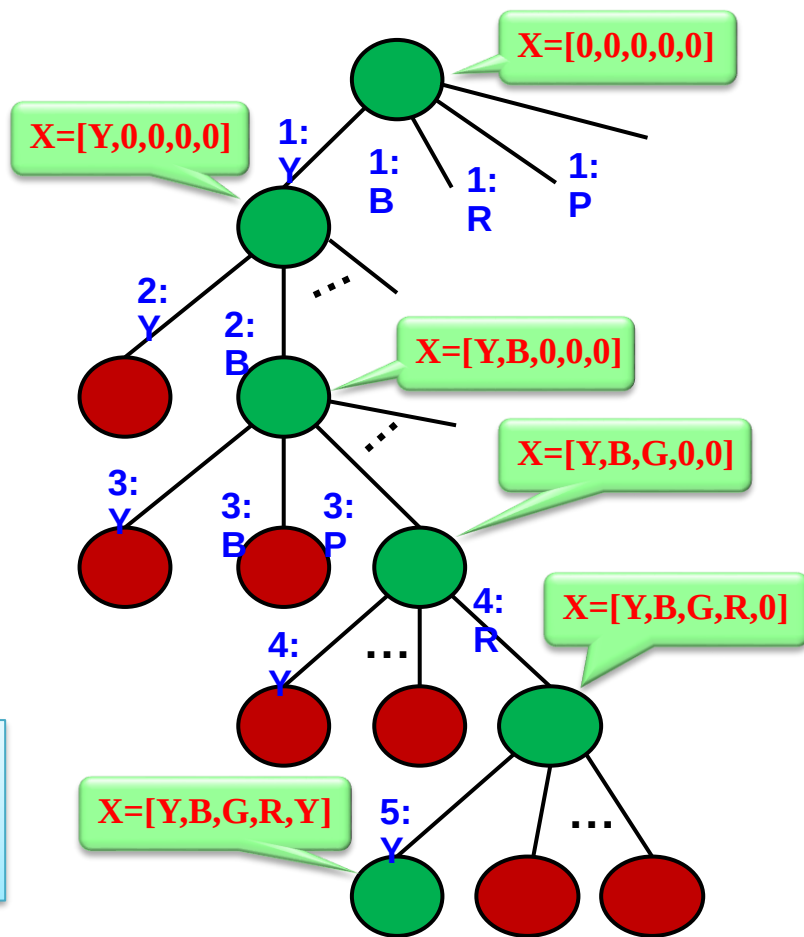


黄色: Y 蓝色: B
绿色: G 红色: R

可行解 (不唯一)

$X=[Y,B,G,R,Y]$

$\vdots$



// 检查顶点v是否可以染颜色c

```
int is_safe(int v, int c, int* graph, int* colors, int n) {  
    for (int i = 0; i < n; i++) {  
        // 检查相邻顶点是否使用相同颜色, graph[]为邻接矩阵（一维数组表示）  
        if (graph[v * n + i] == 1 && colors[i] == c) {  
            return 0;  
        }  
    }  
    return 1;  
}
```

// 回溯法递归函数

```
int backtrack(int v, int n, int m, int* graph, int* colors) {  
    // 所有顶点着色完成  
    if (v == n) return 1;  
  
    // 尝试所有颜色  
    for (int c = 1; c <= m; c++) {  
        if (is_safe(v, c, graph, colors, n)) {  
            colors[v] = c;  
            // 递归处理下一个顶点  
            if (backtrack(v + 1, n, m, graph, colors)) {  
                return 1;  
            }  
            // 回溯  
            colors[v] = 0;  
        }  
    }  
    return 0;  
}
```

回溯法效率分析

➤ 回溯算法的效率在很大程度上依赖于以下因素：

- (1) 产生 $x[k]$ 的时间；
- (2) 满足显约束的 $x[k]$ 值的个数；
- (3) 计算约束函数constraint的时间；
- (4) 计算上界函数bound的时间；
- (5) 满足约束函数和上界函数约束的所有 $x[k]$ 的个数。

➤ 约束

好的约束函数能显著地减少所生成的结点数，但计算量较大。因此，在选择约束函数时通常存在生成结点数与约束函数计算量之间的折衷。

回溯法的时间复杂度一定是指数级的。

☐ A 正确

☒ B 错误

提交

在解决N皇后问题时，以下哪种操作属于剪枝策略？

- ☐ A 每行只放置一个皇后
- ☐ B 记录已放置皇后的行号
- ☒ C 检查当前皇后的位置是否与已放置的皇后存在对角线冲突
- ☐ D 随机选择一个未放置的列位置

提交

作业：

- 1、简述回溯算法中主要的剪枝策略。
- 2、简述回溯法中常见的两种类型的解空间树。
- 3、鸡兔同笼问题是一个笼子里面有鸡和兔子若干只，数一数，共有 a 个头， b 条腿，求鸡和兔子各有多少只？
假设 $a=3$ ， $b=8$ ，画出对应的解空间树。

谢谢 THANKS